

SQL

Introduzione a SQL

Cos'è SQL, dove si usa e come iniziare a leggere i dati.

Perché ti serve SQL?

Ogni volta che cerchi un prodotto in un negozio online, controlli il saldo del conto o apri lo storico degli ordini, qualcuno sta chiedendo dati a un archivio.

SQL è il linguaggio che usi per fare queste richieste. Ti permette di leggere, aggiungere, modificare e cancellare informazioni salvate in un database.

Pensa a SQL come al modo educato e preciso con cui parli a un archivista. Tu fai una domanda chiara. L'archivista cerca nei cassetti giusti e ti restituisce solo quello che hai chiesto.

Cos'è SQL?

SQL sta per **Structured Query Language**. In italiano possiamo leggerlo come "linguaggio per fare richieste organizzate ai dati".

Un database è come un insieme di tabelle, simili a fogli di calcolo. Ogni tabella raccoglie un tipo di informazione: clienti, prodotti, ordini, prenotazioni, pagamenti.

Con SQL puoi dire cose come:

- mostrami tutti i clienti di Roma
 - aggiungi un nuovo prodotto
 - cambia l'indirizzo di un cliente
 - elimina una prenotazione annullata
 - collega gli ordini ai clienti che li hanno fatti
-

Dove si usa SQL nel mondo reale?

SQL è dietro a moltissimi strumenti quotidiani. Anche quando non lo vedi, spesso lavora sotto la superficie.

Lo trovi in:

- **e-commerce**: per cercare prodotti, ordini e clienti
- **app di prenotazione**: per controllare disponibilità e calendari
- **banche**: per consultare movimenti e saldi
- **gestionali aziendali**: per archiviare clienti, fatture e magazzino
- **dashboard e report**: per riassumere dati e numeri importanti
- **app web**: per salvare account, messaggi e impostazioni

SQL vs Python e C++

Python e C++ ti aiutano a scrivere programmi che fanno azioni. SQL ha un compito diverso: **ti aiuta a parlare con i dati**.

Linguaggio	A cosa serve soprattutto
Python	Scrivere programmi, automazioni, analisi e app
C++	Scrivere programmi veloci e vicini alla macchina
SQL	Leggere e modificare dati in un database

Non devi scegliere tra loro. Spesso lavorano insieme. Un'app scritta in Python o C++ può usare SQL per salvare e recuperare informazioni.

Come funziona SQL?

SQL funziona attraverso comandi chiamati **query**. Una query è una domanda o un'istruzione che mandi al database.

Il flusso di lavoro è semplice:

1. scrivi una query
2. il database la legge
3. il database cerca o modifica i dati richiesti
4. tu ricevi un risultato, oppure una conferma dell'operazione

È come compilare un modulo molto preciso. Se scrivi bene la richiesta, il database sa esattamente cosa fare.

La prima query

Per tradizione, in SQL si parte spesso da `SELECT`, il comando che legge i dati.

Questa query mostra tutti i dati della tabella `clienti`:

```
SELECT *  
FROM clienti;
```

In parole semplici significa: **"mostrami tutte le colonne di tutte le righe nella tabella `clienti`".**

Il simbolo `*` vuol dire "tutto". Più avanti imparerai anche a chiedere solo le colonne che ti servono.

Quale SQL useremo?

Useremo soprattutto SQL standard. Questo ti aiuta a capire i concetti senza dipendere subito da un database preciso.

Nella pratica esistono diversi database, come PostgreSQL, MySQL, SQLite e SQL Server. Sono come dialetti della stessa lingua: si assomigliano molto, ma a volte usano parole o dettagli diversi.

Quando una differenza conta davvero, la segnaleremo in modo chiaro.

Il percorso più semplice per iniziare

Se stai partendo da zero, segui questo ordine:

1. Cos'è SQL
2. Database relazionali
3. Tipi di dati
4. CREATE DATABASE
5. CREATE TABLE
6. INSERT, SELECT, WHERE, UPDATE, DELETE
7. GROUP BY, JOIN, Subquery, CTE

8. i temi più avanzati come indici, transazioni e performance

Non devi leggere tutto in una volta. SQL si impara meglio come una cassetta degli attrezzi: oggi un attrezzo, domani un altro.

Come è organizzata questa guida

Qui seguiamo una regola precisa: un concetto per file. Questo rende la documentazione più facile da studiare e più comoda quando vuoi tornare su un argomento specifico.

In pratica, se vuoi capire solo `WHERE`, hai una pagina per `WHERE`. Se vuoi capire solo `JOIN`, trovi una pagina dedicata a `JOIN`.

Da portare a casa

SQL è il linguaggio per fare domande ai dati. All'inizio ti basta imparare poche parole chiave: `SELECT`, `FROM`, `WHERE`, `INSERT`, `UPDATE` e `DELETE`.

Hai già visto il primo passo. Da qui in poi costruiremo il resto con calma, una query alla volta.

Guide consigliate

Per costruire basi solide, continua con:

1. Cos'è SQL
2. Database relazionali
3. Tipi di dati
4. `CREATE DATABASE`
5. `CREATE TABLE`
6. `INSERT`
7. `SELECT`
8. `WHERE`

Poi puoi affrontare `JOIN`, `GROUP BY`, indici e transazioni.

Query ricorsive

CTE che si richiamano da sole per esplorare strutture a livelli.

Qui una query costruisce il passo successivo da sola

Le query ricorsive servono quando i dati hanno una struttura a livelli. Per esempio:

- cartelle e sottocartelle
- categorie e sotto-categorie
- dipendenti e responsabili

In questi casi non basta guardare solo un livello. Devi poter salire o scendere nella gerarchia.

```
WITH RECURSIVE numeri AS (  
    SELECT 1 AS n  
    UNION ALL  
    SELECT n + 1  
    FROM numeri  
    WHERE n < 5  
)  
SELECT * FROM numeri;
```

Questo esempio non usa una gerarchia vera, ma mostra il meccanismo: la query parte da un valore iniziale e poi genera il successivo finché la condizione resta valida.

Perché si chiamano ricorsive

Si chiamano così perché **una parte della query usa il risultato generato dalla query stessa**. È un'idea potente, ma richiede ordine mentale.

Per chi inizia, la chiave è questa: c'è un punto di partenza e c'è una regola che produce il passo seguente.

Quando diventano preziose

Sono molto utili quando vuoi percorrere un albero o una gerarchia senza farlo manualmente riga per riga.

All'inizio possono sembrare avanzate, ma con la giusta immagine mentale diventano molto più comprensibili.

Set operations

UNION, UNION ALL, INTERSECT ed EXCEPT per combinare risultati di query diverse.

Qui lavori su insiemi di risultati

Le set operations permettono di combinare il risultato di più query. Invece di pensare riga per riga, qui ragioni per insiemi.

È un po' come mettere insieme due liste, tenere solo gli elementi in comune o togliere una lista dall'altra.

- **UNION** : unisce senza duplicati
- **UNION ALL** : unisce con duplicati
- **INTERSECT** : tiene solo ciò che coincide
- **EXCEPT** : toglie il secondo insieme dal primo

```
SELECT email FROM clienti_attivi
UNION
SELECT email FROM newsletter;
```

Questa query unisce le email provenienti da due fonti diverse, rimuovendo i duplicati.

La differenza più importante da ricordare

UNION e **UNION ALL** si assomigliano, ma non fanno la stessa cosa:

- **UNION** elimina i duplicati
- **UNION ALL** conserva tutto

Se vuoi tenere anche i ripetuti, scegli **UNION ALL** .

Una regola tecnica da non dimenticare

Per combinare query con queste operazioni, **le colonne restituite devono essere compatibili** per numero e tipo.

In pratica, le due query devono parlare la stessa lingua in uscita.

Window functions

ROW_NUMBER, RANK, DENSE_RANK e OVER per fare classifiche e confronti riga per riga.

Qui guardi il gruppo senza schiacciarlo in una sola riga

Le window functions sono funzioni molto utili quando vuoi analizzare un gruppo di righe, ma senza trasformarlo in una sola riga come fa **GROUP BY**.

Sono perfette per classifiche, posizioni e confronti tra righe vicine.

```
SELECT
  nome,
  totale,
  ROW_NUMBER() OVER (ORDER BY totale DESC) AS posizione
FROM ordini;
```

Questa query assegna un numero di posizione a ogni riga, ordinando i risultati per **totale** dal più grande al più piccolo.

Le funzioni più famose

Funzioni famose:

- **ROW_NUMBER()**
- **RANK()**
- **DENSE_RANK()**

Il ruolo di **OVER**

La parte **OVER (...)** dice alla funzione **su quale finestra di righe deve lavorare** e con quale ordine.

Senza **OVER**, questa famiglia di funzioni non saprebbe su quale insieme ragionare.

Quando sono utili

Le window functions servono spesso per:

- creare classifiche
- numerare le righe
- confrontare risultati vicini
- aggiungere analisi senza perdere il dettaglio riga per riga

Sono un tema un po' più avanzato, ma molto importante quando inizi a fare analisi più ricche.

Funzioni aggregate

COUNT, SUM, AVG, MIN e MAX per riassumere molte righe in un solo risultato.

Quando non vuoi vedere ogni riga

A volte non ti interessa leggere tutti i dati uno per uno. Vuoi un riassunto.

Per esempio:

- quanti ordini ci sono?
- quanto abbiamo incassato?
- qual è il prezzo più alto?
- qual è la media dei voti?

Le **funzioni aggregate** servono proprio a questo: prendono molte righe e restituiscono un risultato riassuntivo.

Contare le righe con COUNT

Questa query conta quante righe ci sono nella tabella `ordini` :

```
SELECT COUNT(*)  
FROM ordini;
```

Se la tabella contiene 125 ordini, il risultato sarà un numero solo: `125` .

`COUNT(*)` significa "conta le righe".

Le funzioni più usate

Ecco le aggregate che incontrerai più spesso:

Funzione	Cosa fa
COUNT ()	conta righe o valori
SUM ()	somma numeri
AVG ()	calcola una media
MIN ()	trova il valore più basso
MAX ()	trova il valore più alto

Alcuni esempi

Per sommare il totale degli ordini:

```
SELECT SUM(totale)
FROM ordini;
```

Per calcolare il prezzo medio dei prodotti:

```
SELECT AVG(prezzo)
FROM prodotti;
```

Per trovare il prezzo più alto:

```
SELECT MAX(prezzo)
FROM prodotti;
```

Per trovare il prezzo più basso:

```
SELECT MIN(prezzo)
FROM prodotti;
```

Ogni query restituisce un solo valore, perché stai chiedendo un riassunto.

Dare un nome al risultato

Il nome automatico della colonna può essere poco leggibile. Per questo puoi usare **AS** :

```
SELECT COUNT(*) AS numero_ordini
FROM ordini;
```

Ora il risultato avrà una colonna chiamata **numero_ordini** .

È un dettaglio piccolo, ma rende le query molto più chiare, soprattutto nei report.

Attenzione a NULL

NULL significa "dato mancante".

Le aggregate lo trattano in modo particolare:

- **COUNT(*)** conta le righe, anche se alcune colonne hanno valori mancanti.
- **COUNT(colonna)** conta solo le righe in cui quella colonna ha un valore.
- **SUM()** , **AVG()** , **MIN()** e **MAX()** di solito ignorano i valori **NULL** .

La cosa importante da ricordare è questa: **mancante non significa zero**.

Riepilogo rapido

- Le funzioni aggregate riassumono molte righe.
- **COUNT(*)** conta le righe.
- **SUM()** somma, **AVG()** calcola la media.
- **MIN()** e **MAX()** trovano il valore più basso e più alto.
- Con **AS** puoi dare un nome leggibile al risultato.

CAST e CONVERT

Cambiare il tipo di un valore quando serve adattarlo al contesto.

A volte il valore è giusto, ma la forma no

Capita di avere un valore corretto nel contenuto, ma nel tipo sbagliato per l'operazione che vuoi fare. In questi casi puoi convertirlo.

`CAST` e, in alcuni database, `CONVERT`, servono proprio a cambiare il tipo di un valore.

```
SELECT CAST(totale AS INT)
FROM ordini;
```

Questa query prende il valore `totale` e lo converte in intero.

Perché può servire

Le conversioni sono utili quando:

- devi confrontare valori che arrivano con tipi diversi
- vuoi formattare un risultato
- vuoi troncare o adattare un valore
- stai lavorando con dati importati in modo poco uniforme

CAST e CONVERT non sono gemelli perfetti

CAST è più standard e più portabile. Se vuoi scrivere SQL che assomigli il più possibile a quello universale, di solito è la scelta migliore.

CONVERT esiste in diversi dialetti, ma non sempre con la stessa sintassi. Per questo conviene usarlo sapendo su quale database ti trovi.

Funzioni su date e tempo

Lavorare con date, orari e timestamp in modo più preciso.

Il tempo nei database conta tantissimo

Date e orari compaiono ovunque. Ordini, iscrizioni, appuntamenti, scadenze, ultime modifiche: quasi ogni applicazione ha bisogno di ricordare quando è successo qualcosa.

Per questo SQL offre funzioni dedicate al tempo.

```
SELECT CURRENT_DATE;
```

```
SELECT CURRENT_TIMESTAMP;
```

La prima query restituisce la data di oggi. La seconda restituisce data e ora correnti.

Un uso tipico: estrarre una parte della data

Molti database permettono di estrarre anno, mese o giorno da una data:

```
SELECT EXTRACT(YEAR FROM data_ordine)
FROM ordini;
```

Qui stai chiedendo solo l'anno della colonna `data_ordine`.

Quando ti tornano utili

Le funzioni su date e tempo servono spesso per:

- filtrare ordini recenti
- raggruppare per mese o anno
- calcolare scadenze
- capire quando un dato è stato creato o modificato

Appena lavori con dati reali, queste funzioni smettono subito di essere opzionali.

GROUP BY

Come raggruppare le righe che condividono lo stesso valore.

Ragionare per gruppi

`GROUP BY` serve quando non vuoi guardare le righe una per una, ma vuoi **raggrupparle**.

Immagina una pila di schede clienti. Invece di leggerle in ordine, le dividi per città: Roma da una parte, Milano da un'altra, Torino da un'altra ancora.

SQL fa la stessa cosa con `GROUP BY`.

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta;
```

Questa query dice:

"Raggruppa i clienti per città e conta quanti clienti ci sono in ogni città."

Un esempio con una tabella

Partiamo da questi dati:

nome	città
Luca	Roma
Marta	Milano
Sara	Roma

La query:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta;
```

produce:

citta	numero_clienti
Milano	1
Roma	2

SQL crea un gruppo per ogni città. Poi `COUNT(*)` conta quante righe ci sono in ciascun gruppo.

Perché va spesso insieme alle aggregate

`GROUP BY` da solo crea i gruppi. Le funzioni aggregate fanno i calcoli dentro quei gruppi.

Per esempio:

- `COUNT(*)` conta quante righe ci sono nel gruppo.
- `SUM(totale)` somma i totali del gruppo.
- `AVG(prezzo)` calcola la media del gruppo.

Per questo `GROUP BY` è così usato nei report.

Una regola da ricordare

Quando usi `GROUP BY`, nel `SELECT` metti di solito:

- la colonna con cui stai raggruppando
- una o più funzioni aggregate

Questa query è chiara:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta;
```

Questa invece crea confusione:

```
SELECT citta, nome, COUNT(*)  
FROM clienti  
GROUP BY citta;
```

Il problema è `nome`. Se a Roma ci sono Luca e Sara, quale nome dovrebbe mostrare SQL per il gruppo "Roma"?

Quando raggruppi, stai lasciando il dettaglio delle singole righe. Per questo devi mostrare il gruppo e i calcoli sul gruppo.

Raggruppare per più colonne

Puoi anche raggruppare usando più colonne:

```
SELECT citta, stato, COUNT(*) AS numero_clienti  
FROM clienti  
GROUP BY citta, stato;
```

Qui SQL crea un gruppo per ogni combinazione di `citta` e `stato`.

È come dividere prima le schede per città e poi, dentro ogni città, separarle per stato.

Riepilogo rapido

- `GROUP BY` raggruppa righe con lo stesso valore.
- Si usa spesso con `COUNT()`, `SUM()`, `AVG()`, `MIN()` e `MAX()`.
- Nel `SELECT` mostri le colonne di raggruppamento e i calcoli aggregati.
- Puoi raggruppare anche per più colonne.

HAVING

Come filtrare i gruppi creati da GROUP BY dopo averli costruiti.

Filtrare dopo il raggruppamento

HAVING assomiglia a **WHERE**, ma lavora in un momento diverso.

WHERE filtra le righe prima di creare i gruppi. **HAVING** filtra i gruppi dopo che **GROUP BY** li ha creati.

Immagina di dividere gli studenti per classe. Prima crei i gruppi. Poi tieni solo le classi con almeno 20 studenti. Quel secondo filtro è il lavoro di **HAVING**.

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta
HAVING COUNT(*) >= 2;
```

Questa query mostra solo le città con almeno due clienti.

Un esempio piccolo

Partiamo da questa tabella:

nome	citta
Luca	Roma
Sara	Roma
Marta	Milano

La query:

```
SELECT citta, COUNT(*) AS numero_clienti
FROM clienti
GROUP BY citta
HAVING COUNT(*) >= 2;
```

produce:

citta	numero_clienti
Roma	2

Milano non appare perché il suo gruppo contiene un solo cliente.

WHERE e HAVING insieme

Puoi usare entrambi nella stessa query:

```
SELECT citta, COUNT(*) AS numero_clienti_attivi
FROM clienti
WHERE attivo = TRUE
GROUP BY citta
HAVING COUNT(*) >= 2;
```

Qui succedono due cose:

1. `WHERE attivo = TRUE` tiene solo i clienti attivi.
2. `HAVING COUNT(*) >= 2` tiene solo le città con almeno due clienti attivi.

Prima filtri le righe. Poi crei i gruppi. Poi filtri i gruppi.

Quando ti serve HAVING

Usa `HAVING` quando la condizione riguarda un risultato aggregato:

- città con più di 10 clienti
- prodotti con vendite sopra una certa soglia
- mesi con almeno 100 ordini
- categorie con prezzo medio superiore a 50

Se nella condizione compare `COUNT()`, `SUM()`, `AVG()` o una funzione simile, probabilmente ti serve `HAVING`.

Riepilogo rapido

- `WHERE` filtra righe singole.
- `HAVING` filtra gruppi.
- `HAVING` si usa dopo `GROUP BY`.
- Si usa spesso con funzioni aggregate come `COUNT()`, `SUM()` e `AVG()`.

Funzioni su stringhe

Operazioni comuni sui testi, come maiuscole, minuscole e concatenazione.

Non sempre il testo va lasciato così com'è

Le stringhe sono i valori testuali. Nomi, email, città, codici, descrizioni: in un database il testo è ovunque.

SQL offre funzioni che ti aiutano a trasformarlo o combinarlo.

```
SELECT UPPER(nome) FROM clienti;
```

Questa query trasforma i nomi in maiuscolo.

Alcune funzioni che vedrai spesso

Funzioni comuni:

- `UPPER()`
- `LOWER()`
- `LENGTH()`
- `TRIM()`
- `CONCAT()`

Due esempi che parlano da soli

Per togliere spazi inutili:

```
SELECT TRIM(nome)  
FROM clienti;
```

Per unire due pezzi di testo:

```
SELECT CONCAT(nome, ' - ', citta)
FROM clienti;
```

Questa seconda query costruisce una piccola etichetta, per esempio `Luca - Roma`.

Quando servono davvero

Le funzioni sulle stringhe sono utili quando:

- vuoi pulire dati inseriti male
- vuoi confrontare testi in modo più uniforme
- vuoi costruire un'etichetta leggibile
- vuoi fare piccoli controlli sulla lunghezza del contenuto

Sono strumenti piccoli, ma diventano molto preziosi nella pratica quotidiana.

ALTER TABLE

Come cambiare una tabella esistente senza ricrearla da zero.

I progetti cambiano, e le tabelle con loro

È raro che una struttura resti perfetta per sempre. All'inizio di un progetto fai una prima scelta. Poi arrivano nuove esigenze, nuovi campi, nomi migliori o regole più precise.

`ALTER TABLE` serve proprio a modificare una tabella che esiste già.

Aggiungere una colonna

Uno degli usi più comuni è questo:

```
ALTER TABLE clienti
ADD telefono VARCHAR(20);
```

Qui stai dicendo al database di aggiungere una nuova colonna chiamata `telefono` .

Rinominare una colonna

A volte il problema non è il contenuto, ma il nome.

```
ALTER TABLE clienti  
RENAME COLUMN telefono TO numero_telefono;
```

Con questa modifica rendi la colonna più chiara senza dover ricreare la tabella intera.

Altri cambiamenti possibili

A seconda del database, `ALTER TABLE` può servire anche per:

- eliminare una colonna
- cambiare un tipo di dato
- aggiungere un vincolo
- togliere una regola esistente
- rinominare la tabella

L'idea centrale resta sempre la stessa: stai facendo manutenzione sulla struttura.

Perché conviene essere prudenti

Quando modifichi una tabella vuota, il rischio è basso. Quando modifichi una tabella piena di dati, la faccenda si fa più delicata.

Per esempio, cambiare tipo a una colonna può creare problemi se i vecchi valori non si adattano bene. Eliminare una colonna significa perdere tutti i dati che conteneva.

Una buona abitudine operativa

Prima di fare un `ALTER TABLE` , è utile:

1. capire bene l'effetto della modifica
2. verificare se altre query o applicazioni dipendono da quella tabella
3. provare la modifica in un ambiente di test

4. fare un backup quando il contesto lo richiede

:::caution `ALTER TABLE` è indispensabile nella vita reale, ma proprio perché tocca la struttura conviene usarlo con metodo. :::

CREATE DATABASE

Come creare il contenitore principale che ospiterà tabelle e dati.

Prima esiste il contenitore, poi il contenuto

Un database è il contenitore generale dei tuoi dati. Prima di creare tabelle, colonne e righe, devi avere un posto in cui tutto questo possa vivere.

È simile a quando prepari una cartella principale prima di riempirla di documenti e sottocartelle.

La sintassi più semplice

```
CREATE DATABASE negozio;
```

Con questa istruzione chiedi al server di creare un database chiamato `negozio`.

Cosa NON fa questo comando

`CREATE DATABASE` non crea ancora tabelle e non salva ancora clienti, prodotti o ordini. Prepara solo il contenitore vuoto.

Dopo questo comando, di solito il passo successivo è collegarti a quel database e iniziare a definirne la struttura.

Quando si usa davvero

Questo comando compare spesso quando:

- inizi un progetto nuovo
- prepari un ambiente di sviluppo o test

- separi i dati di più applicazioni

In molte situazioni quotidiane viene eseguito poche volte. Dopo la fase iniziale, il lavoro si sposta soprattutto su tabelle e query.

Una differenza pratica tra sistemi

Non tutti i database trattano questo comando nello stesso modo. In PostgreSQL e MySQL è del tutto normale creare database con un comando esplicito.

In SQLite, invece, il database spesso coincide con un file. Il concetto è simile, ma il modo pratico di crearlo cambia.

Da portare a casa

Ricorda questa idea: **il database è la casa, le tabelle sono le stanze**. Con `CREATE DATABASE` stai costruendo la casa, non stai ancora arredando le stanze.

CREATE TABLE

Come definire una tabella, le sue colonne e le prime regole di base.

Creare la struttura di una tabella

`CREATE TABLE` serve a creare una nuova tabella.

Non stai ancora inserendo dati. Stai preparando lo spazio che li conterrà: scegli il nome della tabella, le colonne e alcune regole di base.

È come preparare un modulo prima di farlo compilare. Decidi quali campi servono: nome, email, data di iscrizione.

Un primo esempio

Questa query crea una tabella per salvare clienti:

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  email VARCHAR(150),  
  data_iscrizione DATE  
);
```

Dopo questa query, la tabella `clienti` esiste, ma è vuota. Per aggiungere righe userai `INSERT`.

Leggiamola con calma

Dentro le parentesi trovi le colonne:

- `id` è un numero intero e identifica ogni cliente.
- `nome` è testo fino a 100 caratteri e non può mancare.
- `email` è testo fino a 150 caratteri.
- `data_iscrizione` contiene una data.

La tabella, tradotta in italiano, dice:

“Ogni cliente ha un id, un nome, forse un’email e forse una data di iscrizione.”

Quando una tabella si riesce a leggere così, sei sulla strada giusta.

La chiave primaria

`PRIMARY KEY` indica la **chiave primaria**, cioè il valore che identifica una riga senza confonderla con altre.

È come il numero di tessera di una palestra. Due persone possono chiamarsi entrambe Luca, ma non dovrebbero avere lo stesso numero di tessera.

Per questo spesso si usa una colonna `id`: non descrive la persona, la identifica.

Colonne obbligatorie con **NOT NULL**

`NOT NULL` significa che una colonna non può restare vuota.

Nel nostro esempio:

```
nome VARCHAR(100) NOT NULL
```

vuol dire che ogni cliente deve avere un nome.

Suggerimento: Usa `NOT NULL` per i dati davvero necessari. Se un'informazione può mancare nella vita reale, non renderla obbligatoria solo per abitudine.

Tipi e regole

Quando crei una colonna, scegli almeno due cose:

- il **tipo**, cioè che forma avrà il dato
- le **regole**, cioè cosa il database deve controllare

Per esempio:

```
prezzo DECIMAL(10, 2) NOT NULL
```

Qui `DECIMAL(10, 2)` dice che `prezzo` è un numero decimale. `NOT NULL` dice che il prezzo deve esserci.

Il database non conserva soltanto i dati. Ti aiuta anche a tenerli ordinati e coerenti.

Scegliere colonne chiare

All'inizio può venire voglia di creare colonne generiche come `info` o `dati`.

Meglio evitarlo. Una tabella diventa più utile quando ogni colonna ha un compito chiaro.

```
CREATE TABLE prodotti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL,  
  prezzo DECIMAL(10, 2) NOT NULL  
);
```


Qui capisci subito cosa contiene ogni colonna. Non devi indovinare.

Prima di creare una tabella

Fai queste domande:

- che cosa rappresenta ogni riga?
- quale colonna identifica una riga?
- quali dati sono obbligatori?
- quali dati possono mancare?
- i tipi scelti sono adatti ai valori reali?

Non serve progettare tutto perfettamente al primo colpo. Però queste domande ti evitano molte tabelle confuse.

Riepilogo rapido

- `CREATE TABLE` crea la struttura di una tabella.
 - Ogni colonna ha un nome e un tipo.
 - `PRIMARY KEY` identifica ogni riga.
 - `NOT NULL` rende una colonna obbligatoria.
 - Dopo aver creato la tabella, puoi aggiungere dati con `INSERT`.
-

Tipi di dati in SQL

Come scegliere il tipo giusto per numeri, testi, date e valori logici.

Ogni colonna ha bisogno della forma giusta

Quando crei una colonna, devi decidere che cosa potrà contenere. È come scegliere il contenitore corretto prima di sistemare gli oggetti in cucina.

Un barattolo per il sale non è la scelta migliore per conservare la pasta. Allo stesso modo, una colonna pensata per una data non dovrebbe contenere una frase lunga.

Alcuni tipi che userai spesso

Tipo	Significato semplice	Esempio
INT	numero intero	42
VARCHAR(100)	testo corto	'Luca'
TEXT	testo lungo	una descrizione
DATE	data	2026-04-24
BOOLEAN	vero o falso	TRUE
DECIMAL(10,2)	numero decimale preciso	19.99

Perché la scelta conta davvero

Il tipo di dato non è un dettaglio estetico. Aiuta il database a capire come salvare l'informazione, come confrontarla e come difenderla da errori banali.

Per esempio, una colonna prezzo va trattata in modo diverso da una colonna nome. Una colonna data si comporta in modo diverso da un testo normale.

VARCHAR e TEXT : sembrano simili, ma non sono uguali

Quando inizi è normale pensare: "se entrambi tengono testo, perché esistono due tipi?"

- `VARCHAR(100)` dice: **qui entra testo, ma con un limite di lunghezza**
- `TEXT` dice: **qui può entrare anche testo molto lungo**

Per un nome o un'email, spesso `VARCHAR` è una scelta naturale. Per una recensione o una descrizione dettagliata, `TEXT` può avere più senso.

Numeri interi o numeri con virgola

`INT` è comodo per quantità, contatori e identificatori. Se devi salvare quante unità hai in magazzino, è un buon candidato.

`DECIMAL` è più adatto per prezzi, importi e valori dove la precisione conta. Quando ci sono di mezzo soldi, conviene essere precisi.

Un esempio dentro una tabella

Qui sotto vedi una tabella dove i tipi di dato raccontano già molto delle colonne:

```
CREATE TABLE prodotti (  
  id INT,  
  nome VARCHAR(100),  
  prezzo DECIMAL(10,2),  
  disponibile BOOLEAN,  
  data_creazione DATE  
);
```

Leggendo questa struttura capisci subito che `prezzo` è un valore numerico preciso, `disponibile` è un sì/no e `data_creazione` è una data.

...tip Se sei indeciso sul tipo, chiediti: "che aspetto ha questo dato nel mondo reale?" La risposta spesso ti guida bene.
...

DROP TABLE

Come eliminare completamente una tabella e quando farlo con cautela.

Qui non stai riordinando: stai togliendo tutto

DROP TABLE elimina una tabella intera. Non cancella solo le righe. Fa sparire anche la struttura della tabella.

È come gettare via un intero raccoglitore, non solo i fogli che contiene.

Il comando base

```
DROP TABLE clienti;
```

Dopo questo comando la tabella `clienti` non esiste più.

Cosa viene eliminato davvero

Con `DROP TABLE` perdi:

- il nome della tabella
- le sue colonne
- i dati salvati dentro
- eventuali collegamenti o dipendenze da gestire

Per questo è molto diverso da `DELETE`, che toglie righe ma lascia viva la tabella.

Quando può avere senso

Ci sono casi in cui è una scelta legittima:

- hai creato una tabella di prova
- una tabella temporanea non serve più
- stai rifacendo lo schema in ambiente di sviluppo

In un database importante o condiviso, però, questo comando richiede molta attenzione.

Le domande da farti prima

Prima di lanciare il comando, fermati e controlla:

1. voglio davvero eliminare anche la struttura?
2. ci sono altre tabelle che dipendono da questa?
3. esiste un backup o un piano di recupero?

Queste verifiche ti proteggono da errori costosi.

⚠ Se il tuo obiettivo è solo svuotare la tabella ma lasciarla esistente, `DROP TABLE` non è lo strumento giusto. ⚠

Database relazionali

Cosa sono tabelle, righe, colonne e relazioni tra i dati.

Immaginalo come un insieme di fogli collegati

Un database relazionale assomiglia a una raccolta di tabelle. Se hai mai visto un foglio Excel, hai già un'immagine utile in testa.

La differenza importante è questa: qui le tabelle possono collegarsi tra loro in modo ordinato.

I tre pezzi da riconoscere subito

Dentro una tabella trovi sempre:

- una **tabella**: l'intero elenco, per esempio `clienti`
- una **riga**: un elemento preciso, per esempio un cliente
- una **colonna**: un tipo di informazione, per esempio `nome` o `email`

Guarda questa mini tabella:

id	nome	citta
1	Luca	Roma
2	Marta	Milano

Qui ogni riga è una persona. Ogni colonna descrive un aspetto di quella persona.

Perché si usa la parola relazionale

Perché le tabelle possono fare riferimento una all'altra. Per esempio puoi avere:

- una tabella `clienti`
- una tabella `ordini`

Ogni ordine può contenere un collegamento al cliente che lo ha effettuato. In questo modo i dati restano più ordinati e non devi riscrivere tutto ogni volta.

Un'analogia che aiuta davvero

Pensa a una biblioteca.

- un foglio tiene i **lettori**
- un altro foglio tiene i **prestiti**

Nei prestiti non hai bisogno di riscrivere sempre l'indirizzo completo del lettore. Ti basta un riferimento al lettore giusto. **Questo è il cuore del modello relazionale: collegare bene le informazioni senza duplicarle troppo.**

Cosa rende utile questo modello

Un database relazionale aiuta a:

- mantenere i dati più puliti
- evitare ripetizioni inutili
- cercare informazioni con precisione
- combinare dati di tabelle diverse
- applicare regole per ridurre gli errori

Un'anticipazione dei prossimi passi

Più avanti incontrerai termini come `PRIMARY KEY`, `FOREIGN KEY` e `JOIN`. Non serve padroneggiarli subito.

Per ora ti basta questa immagine: **un database relazionale è un insieme di tabelle che collaborano tra loro in modo controllato.**

Cos'è SQL

Una guida semplice per capire cosa sia SQL, perché esiste e in quali situazioni lo usi davvero.

Partiamo da una scena concreta

Immagina di avere un negozio online. Ci sono clienti che comprano, prodotti in vendita, ordini fatti, pagamenti ricevuti, indirizzi per spedire. Tutte queste informazioni devono essere conservate da qualche parte, in modo ordinato e facile da cercare.

Quel "posto" si chiama database. E **SQL è il linguaggio per parlare con quel posto**, per chiedergli le informazioni che ti servono e per aggiungerne di nuove.

:::note Se non hai mai sentito la parola "database", niente paura. Pensa a un armadio con tanti cassetti numerati. Ogni cassetto contiene una cosa diversa: uno ha i nomi dei clienti, un altro ha i prodotti, un altro ancora gli ordini. Il database è proprio così, ma al posto dei fogli di carta usa i dati nel computer. :::

Cosa significa il nome

SQL sta per **Structured Query Language**. Il nome può sembrare impegnativo, ma l'idea è semplice: è un linguaggio fatto per fare richieste strutturate a un insieme di dati.

Una *query* è una richiesta. **Pensa a una query come a una domanda specifica** che fai a un amico: "Quali sono i clienti di Roma?" oppure "Dov'è quel prodotto?". La differenza è che invece di chiedere a un amico, la fai al database.

:::note "Strutturato" significa che i dati sono organizzati in modo ordinato, come una tabella con righe e colonne, non sparsi ovunque. :::

Cosa puoi fare con SQL

Con SQL puoi fare quattro cose principali, come se stessi lavorando su un archivio:

- **leggere** dati già salvati (come cercare un libro in una biblioteca)
- **inserire** nuovi dati (come aggiungere un nuovo libro allo scaffale)
- **aggiornare** dati esistenti (come correggere un errore di ortografia in una scheda)
- **eliminare** dati che non servono più (come buttare un vecchio catalogo)

E poi c'è anche il **collegare** dati che stanno in tabelle diverse — come unire la lista dei clienti con la lista degli ordini per vedere cosa hanno comprato.

:::note In gergo tecnico, le "tabelle" sono come fogli Excel: hanno righe e colonne. Ogni riga è un elemento (un cliente, un prodotto), ogni colonna è un tipo di informazione (nome, prezzo, data). :::

Dove lo incontri nella pratica

SQL è molto più comune di quanto sembri. Lo trovi dietro a tanti strumenti che usi ogni giorno, anche se non te ne accorgi:

- **siti di e-commerce** quando cerchi un prodotto
- **app di prenotazione** quando controlli la disponibilità di un hotel
- **gestionali aziendali** quando cerchi un cliente
- **software bancari** quando controlli il saldo
- **pannelli di amministrazione** quando gestisci un sito web
- **strumenti di analisi** quando generi un report

:::tip La prossima volta che fai una ricerca su un sito e appare un risultato, pensa: "Qualcuno ha fatto una query SQL per trovare questo!" :::

Quando un'app salva o recupera dati strutturati, c'è una buona possibilità che in qualche punto stia usando SQL.

Un esempio piccolo da leggere senza paura

Ora vediamo un esempio pratico. Questa query legge una colonna precisa di una tabella:

```
SELECT nome
FROM clienti;
```

Cosa significa in parole semplici: "Prendi la colonna `nome` dalla tabella `clienti`".

- `SELECT` = "prendi" o "mostra"
- `nome` = la colonna che vuoi vedere
- `FROM` = "da" (indica da dove prendere i dati)
- `clienti` = la tabella che contiene i dati

:::note Le parole chiave come `SELECT` e `FROM` sono sempre in maiuscolo per convenzione, ma SQL non è sensibile alle maiuscole. Puoi scriverle anche in minuscolo. :::

Hai appena letto il tuo primo comando SQL. È un piccolo passo, ma è già il cuore di tutto: fare una domanda chiara ai dati.

Perché vale la pena impararlo

Capire SQL ti rende più forte anche quando usi strumenti più comodi. Se un giorno lavorerai con ORM, dashboard o framework, sapere SQL ti aiuterà a capire cosa succede davvero sotto la superficie.

:::tip Quando una query ti sembra oscura, prova a trasformarla in una frase italiana. Spesso la nebbia si dirada subito. :::

Riassunto veloce

SQL è come un linguaggio per parlare con un database. Lo usi per chiedere informazioni, aggiungerne di nuove, modificarle o cancellarle. E non devi essere un esperto per iniziare: basta conoscere le basi per fare le tue prime richieste.

DELETE

Come eliminare righe da una tabella senza cancellare più del necessario.

Cancellare righe da una tabella

DELETE serve a rimuovere righe da una tabella.

Non elimina la tabella intera. Elimina solo i dati che scegli con il filtro.

Per esempio:

```
DELETE FROM clienti
WHERE id = 3;
```

Questa query cancella il cliente con `id = 3`.

Il filtro è la parte più importante

Con **DELETE**, **WHERE** è la tua cintura di sicurezza.

```
WHERE id = 3
```

Senza questa condizione, il database potrebbe cancellare tutte le righe della tabella.

Attenzione: Prima di eseguire un **DELETE**, controlla sempre quali righe stai per cancellare.

Il controllo con **SELECT**

Il modo più prudente è provare prima la stessa condizione con una **SELECT**:

```
SELECT *
FROM clienti
WHERE id = 3;
```

Se il risultato è proprio la riga che vuoi eliminare, puoi usare lo stesso filtro nel **DELETE**.

Altro esempio:

```
SELECT *  
FROM clienti  
WHERE email = 'test@email.it';
```

Se vedi solo il cliente di test, puoi cancellarlo così:

```
DELETE FROM clienti  
WHERE email = 'test@email.it';
```

La condizione è identica. Cambia solo l'azione.

DELETE non è DROP TABLE

È facile confonderli all'inizio, ma fanno cose molto diverse:

- **DELETE** elimina righe.
- **DROP TABLE** elimina la tabella intera, compresa la struttura.

Se immagini una rubrica, **DELETE** cancella uno o più contatti. **DROP TABLE** butta via tutta la rubrica.

A volte è meglio non cancellare

Nel mondo reale, non sempre conviene eliminare davvero una riga.

Per esempio, se un cliente ha fatto ordini o ricevuto fatture, potresti voler conservare la sua storia. In questi casi spesso si aggiunge una colonna come **attivo**.

```
UPDATE clienti  
SET attivo = FALSE  
WHERE id = 3;
```

Così il cliente non risulta più attivo, ma i suoi dati restano disponibili per controlli futuri.

Riepilogo rapido

- `DELETE` elimina righe.
- `WHERE` decide quali righe eliminare.
- Prima di cancellare, controlla il filtro con una `SELECT`.
- `DELETE` non elimina la struttura della tabella.
- In alcuni casi conviene disattivare una riga invece di cancellarla.

INSERT

Come aggiungere nuove righe a una tabella in modo chiaro e sicuro.

Aggiungere una nuova riga

`INSERT` serve quando vuoi **mettere nuovi dati dentro una tabella**.

Se la tabella è una rubrica, `INSERT` aggiunge un nuovo contatto. Se è un catalogo prodotti, aggiunge un nuovo prodotto.

Ecco un esempio semplice:

```
INSERT INTO clienti (nome, email)
VALUES ('Luca', 'luca@email.it');
```

Puoi leggerlo così:

"Nella tabella `clienti`, riempi le colonne `nome` ed `email` con questi valori."

Il primo valore, `'Luca'`, finisce nella prima colonna indicata, cioè `nome`. Il secondo valore, `'luca@email.it'`, finisce in `email`.

Perché scrivere i nomi delle colonne

SQL permette anche forme più brevi, ma quando stai imparando è meglio indicare sempre le colonne.

Questa forma è poco chiara:

```
INSERT INTO clienti
VALUES (1, 'Luca', 'luca@email.it');
```

Funziona solo se conosci esattamente l'ordine delle colonne nella tabella.

Questa invece si legge meglio:

```
INSERT INTO clienti (id, nome, email)
VALUES (1, 'Luca', 'luca@email.it');
```

Anche tra qualche mese capirai subito cosa stai inserendo e dove.

Inserire più righe insieme

Se devi aggiungere più clienti, puoi farlo con un solo `INSERT` :

```
INSERT INTO clienti (nome, email)
VALUES
  ('Marta', 'marta@email.it'),
  ('Paolo', 'paolo@email.it');
```

Ogni coppia tra parentesi crea una nuova riga.

Quando manca un valore

Non tutte le colonne devono essere sempre compilate.

Se una colonna è facoltativa, puoi non inserirla. Se invece è obbligatoria, per esempio perché ha `NOT NULL` , il database rifiuterà la riga.

Per esempio, se `nome` è obbligatorio, questa query può fallire:

```
INSERT INTO clienti (email)
VALUES ('senza.nome@email.it');
```

Il database ti sta proteggendo da un dato incompleto: non vuole salvare un cliente senza nome.

Attenzione ai tipi di dato

Ogni colonna si aspetta un certo tipo di valore.

```
INSERT INTO prodotti (nome, prezzo, disponibile)
VALUES ('Tastiera', 39.90, TRUE);
```

Qui stai inserendo:

- un testo: 'Tastiera'
- un numero decimale: 39.90
- un valore vero/falso: TRUE

Non tutti i database scrivono i valori booleani nello stesso identico modo, ma l'idea resta la stessa: ogni valore deve essere adatto alla sua colonna.

Prima di eseguire un INSERT

Fai un controllo veloce:

- sto compilando le colonne obbligatorie?
- i valori sono nell'ordine giusto?
- ogni valore ha il tipo corretto?

Sono domande semplici, ma evitano molti errori.

Riepilogo rapido

- **INSERT INTO** aggiunge nuove righe.
- Conviene indicare sempre le colonne.
- **VALUES** contiene i dati da inserire.

- L'ordine dei valori deve seguire l'ordine delle colonne.
- Il database può rifiutare dati mancanti o del tipo sbagliato.

UPDATE

Come modificare dati già presenti nel database senza toccare le righe sbagliate.

Modificare una riga esistente

UPDATE serve quando un dato esiste già, ma deve cambiare.

Un cliente cambia città. Un prodotto cambia prezzo. Un ordine passa da "in preparazione" a "spedito". In tutti questi casi non vuoi creare una nuova riga: vuoi aggiornare quella giusta.

Ecco la forma base:

```
UPDATE clienti
SET citta = 'Bologna'
WHERE id = 1;
```

Questa query dice:

*"Nella tabella **clienti** , cambia la città in **Bologna** , ma solo per la riga con **id = 1** ."*

Il ruolo di **SET**

SET indica cosa deve cambiare:

```
SET citta = 'Bologna'
```

Puoi aggiornare anche più colonne insieme:

```
UPDATE clienti
SET citta = 'Bologna', email = 'luca.nuova@email.it'
WHERE id = 1;
```

Qui cambi sia la città sia l'email dello stesso cliente.

Il ruolo di WHERE

La parte più delicata è WHERE .

```
WHERE id = 1
```

Questa condizione sceglie la riga da modificare. Senza di lei, il database non sa che vuoi aggiornare un solo cliente.

Attenzione: Un UPDATE senza WHERE può modificare tutte le righe della tabella.

Una buona abitudine

Prima di fare un aggiornamento importante, prova la condizione con una SELECT :

```
SELECT *
FROM clienti
WHERE id = 1;
```

Se la SELECT mostra una sola riga, sai che l' UPDATE toccherà una sola riga. Se ne mostra molte, fermati e controlla meglio il filtro.

Usare il valore attuale

A volte il nuovo valore dipende da quello già presente.

Per esempio, questa query aumenta del 10% il prezzo di un prodotto:

```
UPDATE prodotti
SET prezzo = prezzo * 1.10
WHERE id = 5;
```

`prezzo * 1.10` significa: “prendi il prezzo attuale e moltiplicalo per 1,10”.

È una possibilità utile, ma va usata con attenzione. Anche qui, il filtro `WHERE` decide quali righe cambieranno.

Riepilogo rapido

- `UPDATE` modifica righe già esistenti.
- `SET` dice quali colonne cambiare.
- `WHERE` dice quali righe modificare.
- Prima di un aggiornamento importante, controlla il filtro con una `SELECT`.
- Senza `WHERE`, rischi di aggiornare tutta la tabella.

Indici avanzati

Indici composti, full-text, B-tree e hash spiegati senza gergo inutile.

Esistono indici diversi perché i problemi da risolvere sono diversi

Non tutti gli indici funzionano nello stesso modo. Alcuni sono pensati per confronti classici, altri per più colonne insieme, altri ancora per cercare dentro grandi testi.

È come avere strumenti diversi in cucina: un coltello, una grattugia e un colino non fanno lo stesso lavoro. Sono tutti utili, ma solo nel contesto giusto.

Alcuni tipi importanti

- **composti:** su più colonne
- **B-tree:** il tipo più comune
- **hash:** utile in casi specifici

- **full-text**: per cercare dentro grandi testi

Indici composti

Un indice composto riguarda più colonne insieme, per esempio `(cognome, nome)`.

È utile quando le query filtrano o ordinano spesso usando proprio quella combinazione.

```
CREATE INDEX idx_clienti_cognome_nome  
ON clienti(cognome, nome);
```

Questo indice può aiutare quando cerchi spesso per cognome e poi per nome.

B-tree, hash e full-text

B-tree è il tipo più comune e versatile.

hash può aiutare in alcuni confronti di uguaglianza.

full-text è pensato per ricerche testuali più ricche, dove non basta un semplice confronto.

Per esempio, una ricerca full-text può essere utile quando vuoi cercare parole dentro descrizioni lunghe, articoli o commenti.

Un promemoria importante

Ogni database offre opzioni diverse. **Non è necessario imparare tutti i tipi in una volta.** La cosa più utile all'inizio è capire che esistono indici specializzati per problemi diversi.

:::tip Parti dagli indici semplici. Gli indici avanzati hanno senso quando hai già una query reale da migliorare. :::

CREATE INDEX

Come creare un indice per velocizzare certe ricerche nel database.

Un indice è come l'indice finale di un libro

Un indice aiuta il database a trovare più rapidamente certe righe. Senza indice, a volte il database deve scorrere tantissimo contenuto. Con un buon indice, il percorso può diventare molto più rapido.

Immagina un libro di 800 pagine. Se cerchi la parola "spedizione" senza indice, sfogli tutto. Con un indice, vai quasi subito alle pagine giuste.

Nel database succede qualcosa di simile.

Un esempio piccolo

Questa istruzione crea un indice sulla colonna `email` della tabella `clienti` :

```
CREATE INDEX idx_clienti_email  
ON clienti(email);
```

Da quel momento, una ricerca per email può diventare più veloce.

```
SELECT nome  
FROM clienti  
WHERE email = 'luca@email.it';
```

Quando ha senso aggiungerlo

Gli indici sono utili soprattutto su colonne che usi spesso per:

- filtrare con `WHERE`
- ordinare con `ORDER BY`
- collegare tabelle nelle join

Il compromesso da ricordare

Gli indici accelerano alcune letture, ma non sono gratis. Occupano spazio e possono rallentare un po' inserimenti, aggiornamenti ed eliminazioni.

Per questo non si indicizza tutto a caso. Si indicizza ciò che serve davvero.

:::tip Prima di aggiungere tanti indici, osserva quali query sono davvero lente. Un indice scelto bene aiuta. Dieci indici scelti a caso possono creare solo rumore. :::

DROP INDEX

Come eliminare un indice che non sta aiutando o non serve più.

Non tutti gli indici meritano di restare

Un indice utile è prezioso. Un indice inutile è solo peso. Se un indice non porta benefici reali, puoi rimuoverlo.

È come tenere un vecchio indice cartaceo di un archivio che nessuno consulta più. Occupa spazio e va mantenuto, ma non aiuta davvero.

Un esempio piccolo

```
DROP INDEX idx_clienti_email;
```

Questa istruzione elimina l'indice chiamato `idx_clienti_email`.

:::caution Stai eliminando l'indice, non la tabella `clienti` e non i suoi dati. :::

Quando può avere senso farlo

Puoi eliminare un indice quando:

- la query che doveva aiutare non esiste più
- l'indice non viene usato davvero
- il costo di manutenzione supera il vantaggio

Un dettaglio pratico

La sintassi precisa cambia un po' tra i database. L'idea però resta la stessa: **stai togliendo una struttura di supporto, non i dati veri della tabella.**

Prima di rimuovere un indice in un progetto reale, controlla se qualche query importante lo usa ancora. In caso di dubbio, misura prima e cambia dopo.

Join avanzati

SELF JOIN e CROSS JOIN per casi meno comuni ma molto utili.

Quando le join normali non bastano

Non tutte le join servono a collegare due tabelle diverse. A volte devi unire una tabella con se stessa. Altre volte vuoi generare tutte le combinazioni possibili tra due insiemi.

Qui entrano in gioco `SELF JOIN` e `CROSS JOIN`.

SELF JOIN

Con `SELF JOIN` una tabella viene collegata a se stessa.

```
SELECT d.nome AS dipendente, r.nome AS responsabile
FROM dipendenti AS d
LEFT JOIN dipendenti AS r ON d.responsabile_id = r.id;
```

Questo succede perché nella tabella `dipendenti` ogni persona può avere un responsabile che è, a sua volta, un altro dipendente.

CROSS JOIN

`CROSS JOIN` crea tutte le combinazioni possibili tra due tabelle.

```
SELECT colori.nome, taglie.nome
FROM colori
CROSS JOIN taglie;
```

Se hai 3 colori e 4 taglie, il risultato avrà 12 righe.

Quando hanno senso

SELF JOIN è utile per strutture gerarchiche, come:

- dipendente e responsabile
- categoria e sotto-categoria
- persona e referente

CROSS JOIN è utile quando vuoi costruire tutte le combinazioni teoriche, per esempio colori e taglie di un catalogo.

CTE

Usare WITH per creare risultati temporanei leggibili e riutilizzabili nella query.

Dare un nome a un passaggio intermedio

CTE significa **Common Table Expression**. Il nome è tecnico, ma l'idea è semplice: crei un risultato temporaneo, gli dai un nome e lo usi subito dopo.

È come prendere appunti mentre risolvi un problema. Invece di tenere tutto in testa, scrivi un passaggio, lo chiami in modo chiaro e poi continui.

```
WITH ordini_grandi AS (  
    SELECT id, cliente_id, totale  
    FROM ordini  
    WHERE totale > 100  
)  
SELECT id, cliente_id, totale  
FROM ordini_grandi;
```

Qui **ordini_grandi** è un nome temporaneo. Dentro ci sono gli ordini con totale maggiore di 100.

La CTE esiste solo per questa query. Non crea una nuova tabella permanente nel database.

Perché rende le query più leggibili

Senza CTE, una query lunga può diventare difficile da seguire.

Con una CTE, invece, puoi separare il ragionamento:

1. prima preparo un gruppo di dati
2. poi uso quel gruppo nella query principale

Il vantaggio è soprattutto mentale. Leggi la query come una piccola storia.

Un esempio con una join

Troviamo prima gli ordini sopra 100 euro. Poi colleghiamoli ai clienti:

```
WITH ordini_grandi AS (  
    SELECT id, cliente_id, totale  
    FROM ordini  
    WHERE totale > 100  
)  
SELECT clienti.nome, ordini_grandi.totale  
FROM ordini_grandi  
INNER JOIN clienti ON clienti.id = ordini_grandi.cliente_id;
```

La CTE si occupa del primo passaggio: "quali sono gli ordini grandi?".

La query principale si occupa del secondo: "a quali clienti appartengono?".

Più CTE nella stessa query

Puoi anche creare più risultati temporanei:

```
WITH ordini_grandi AS (  
    SELECT id, cliente_id, totale  
    FROM ordini  
    WHERE totale > 100  
)  
,  
clienti_attivi AS (  
    SELECT id, nome  
    FROM clienti  
    WHERE attivo = TRUE  
)  
SELECT clienti_attivi.nome, ordini_grandi.totale  
FROM ordini_grandi  
INNER JOIN clienti_attivi ON clienti_attivi.id = ordini_grandi.cliente_id;
```

Ogni CTE ha un nome e un compito. Questo aiuta molto quando una query cresce.

CTE o subquery?

Una subquery va benissimo quando il passaggio interno è piccolo.

Una CTE diventa più comoda quando vuoi dare un nome al passaggio, riusarlo o rendere la query più leggibile.

Non è una gara tra strumenti. Scegli quello che rende più chiaro il ragionamento.

Riepilogo rapido

- Una CTE nasce con **WITH**.
- Crea un risultato temporaneo con un nome.
- Vive solo per la durata della query.
- Aiuta a leggere query lunghe o divise in più passaggi.
- Spesso è più chiara di una subquery molto annidata.

Join

Come collegare tabelle diverse con INNER JOIN, LEFT JOIN, RIGHT JOIN e FULL JOIN.

Mettere insieme dati che vivono in tabelle diverse

In un database relazionale, le informazioni non stanno sempre tutte nella stessa tabella.

Per esempio, potresti avere una tabella `clienti` e una tabella `ordini`. La prima contiene le persone. La seconda contiene gli acquisti.

Da sole sono utili, ma insieme raccontano una storia più completa: **chi ha comprato cosa**.

Le `JOIN` servono proprio a questo: collegare tabelle diverse mentre leggi i dati.

Un primo esempio

Immagina queste due tabelle.

clienti

id	nome
1	Luca
2	Marta
3	Sara

ordini

id	cliente_id	totale
10	1	49.90
11	2	18.50

Nella tabella `ordini`, la colonna `cliente_id` dice a quale cliente appartiene ogni ordine.

Per mostrare il nome del cliente insieme al totale dell'ordine, scrivi:

```
SELECT clienti.nome, ordini.totale
FROM clienti
INNER JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato sarà:

nome	totale
Luca	49.90
Marta	18.50

Sara non appare perché non ha ordini collegati.

Cosa fa **INNER JOIN**

INNER JOIN tiene solo le righe che trovano una corrispondenza in entrambe le tabelle.

Nel nostro esempio:

- Luca ha un ordine, quindi appare.
- Marta ha un ordine, quindi appare.
- Sara non ha ordini, quindi non appare.

È come fare l'appello usando due liste e tenere solo i nomi che riesci ad abbinare.

La parte più importante: **ON**

Questa riga è il collegamento:

```
ON clienti.id = ordini.cliente_id
```

Significa:

"Abbinare un cliente a un ordine quando l'id del cliente è uguale al `cliente_id` salvato nell'ordine."

Se sbagli questa condizione, la join collega righe sbagliate. Per questo conviene leggerla sempre con calma.

Perché scriviamo **clienti.nome**

Quando usi più tabelle, possono esistere colonne con lo stesso nome.

Per esempio, sia `clienti` sia `ordini` hanno una colonna `id`. Scrivere `clienti.id` o `ordini.id` evita ambiguità.

È come usare nome e cognome quando in una stanza ci sono due persone chiamate Luca.

LEFT JOIN : tenere anche chi non ha corrispondenze

A volte vuoi vedere tutti i clienti, anche quelli che non hanno ancora fatto ordini.

In quel caso usi `LEFT JOIN` :

```
SELECT clienti.nome, ordini.totale
FROM clienti
LEFT JOIN ordini ON clienti.id = ordini.cliente_id;
```

Il risultato diventa:

nome	totale
Luca	49.90
Marta	18.50
Sara	NULL

`NULL` significa che manca un valore. Qui vuol dire: "per Sara non esiste un ordine collegato".

Quale join scegliere?

All'inizio concentrati su queste due:

- `INNER JOIN` quando vuoi solo le righe che si abbinano.
- `LEFT JOIN` quando vuoi tenere tutte le righe della tabella di sinistra.

Esistono anche `RIGHT JOIN` e `FULL JOIN` , ma puoi incontrarle più avanti. Nelle query quotidiane, `INNER JOIN` e `LEFT JOIN` coprono già tantissimi casi.

Riepilogo rapido

- Una `JOIN` legge insieme dati di tabelle diverse.

- `INNER JOIN` mostra solo le corrispondenze.
 - `LEFT JOIN` tiene anche le righe senza corrispondenza a destra.
 - `ON` spiega come collegare le tabelle.
 - Scrivere `tabella.colonna` evita confusione quando più tabelle hanno colonne simili.
-

Subquery

Come usare una query dentro un'altra per costruire richieste a più passaggi.

Una domanda dentro un'altra domanda

Una **subquery** è una query scritta dentro un'altra query.

Ti serve quando la risposta finale dipende da un passaggio intermedio. Prima trovi qualcosa. Poi usi quel risultato per fare la domanda principale.

Immagina di voler trovare i clienti che hanno fatto almeno un ordine sopra 100 euro. Puoi pensarla in due passi:

1. trova gli ordini sopra 100 euro
2. trova i clienti collegati a quegli ordini

In SQL diventa:

```
SELECT nome
FROM clienti
WHERE id IN (
    SELECT cliente_id
    FROM ordini
    WHERE totale > 100
);
```

La query interna è questa:

```
SELECT cliente_id
FROM ordini
WHERE totale > 100
```

Produce una lista di id cliente. La query esterna usa quella lista per trovare i nomi nella tabella `clienti`.

Perché usare una subquery

Una subquery è utile quando il problema si legge naturalmente a tappe.

Per esempio:

- trova i prodotti più costosi della media
- mostra i clienti che hanno fatto ordini
- cerca gli studenti iscritti a un certo corso

In tutti questi casi, una parte della query prepara il risultato per l'altra.

Subquery con una lista

Quando la query interna restituisce più valori, spesso usi `IN`.

```
SELECT nome
FROM clienti
WHERE id IN (
    SELECT cliente_id
    FROM ordini
);
```

Qui stai dicendo:

"Mostrami i clienti il cui id compare nella lista dei `cliente_id` presenti negli ordini."

Subquery con un solo valore

A volte la query interna restituisce un valore solo.

Per esempio, questa query trova i prodotti che costano più della media:

```
SELECT nome, prezzo
FROM prodotti
WHERE prezzo > (
    SELECT AVG(prezzo)
    FROM prodotti
);
```

La query interna calcola la media dei prezzi. La query esterna mostra solo i prodotti con prezzo superiore a quella media.

Qui non usi **IN**, perché non stai confrontando con una lista. Stai confrontando con un solo numero.

Quando preferire una CTE

Le subquery sono comode, ma possono diventare difficili da leggere quando si annidano troppo.

Se senti che la query sta diventando una matrioska, spesso una CTE con **WITH** è più chiara. Le vediamo nella pagina dedicata.

Riepilogo rapido

- Una subquery è una query dentro un'altra query.
- Serve quando una domanda ha bisogno di un passaggio intermedio.
- Con **IN** confronti un valore con una lista.
- Con operatori come **>**, **<** o **=** confronti con un singolo valore.
- Se la query diventa troppo annidata, valuta una CTE.

Normalizzazione 1NF

La prima forma normale è il principio di un valore per cella.

La prima regola dell'ordine è: una casella, un solo valore

La prima forma normale, detta 1NF, dice che ogni cella deve contenere un solo valore. Niente elenchi nascosti dentro una stessa colonna.

Pensa a una rubrica cartacea. Se nella casella "telefono" scrivi tre numeri separati da virgole, poi diventa difficile cercare, correggere o cancellare un solo numero.

Non bene:

id	nome	telefoni
1	Luca	123, 456

Qui la colonna `telefoni` contiene due valori insieme. Questo rende più difficile cercare, aggiornare e controllare i dati.

Come sistemarla

Hai due strade migliori:

- creare una riga per ogni telefono in una tabella separata
- oppure ripensare la struttura se il dato è stato modellato male

La 1NF è la base della pulizia. Se la salti, il resto del database parte già storto.

Meglio:

cliente_id	telefono
1	123
1	456

Ora ogni cella contiene un solo valore. Cercare il telefono `456` diventa molto più semplice.

:::note La 1NF non rende il database perfetto da sola. Però evita uno degli errori più comuni: infilare liste dentro una singola cella. :::

Normalizzazione 2NF

La seconda forma normale è l'idea di dipendere dall'intera chiave.

Qui entri in un concetto un po' più sottile

La seconda forma normale, o 2NF, dice che ogni colonna deve dipendere dall'intera chiave primaria, non solo da una parte.

Questo tema compare soprattutto quando la chiave primaria è composta da più colonne.

Pensa a una ricevuta con due informazioni che identificano una riga: `ordine_id` e `prodotto_id`. Il prezzo comprato in quella riga dipende da quella combinazione. Il nome del prodotto, invece, dipende solo dal prodotto.

Un modo semplice di pensarla

Se una tabella usa una chiave composta, ogni informazione non chiave dovrebbe avere senso solo guardando **tutta** la chiave.

Se basta un pezzo della chiave per determinare quel valore, probabilmente quel dato starebbe meglio in un'altra tabella.

Un esempio intuitivo

Immagina questa tabella:

<code>ordine_id</code>	<code>prodotto_id</code>	<code>nome_prodotto</code>	<code>quantita</code>
10	3	Tastiera	2

La quantità dipende dall'ordine e dal prodotto insieme. Il `nome_prodotto`, invece, dipende solo da `prodotto_id`.

In 2NF, il nome del prodotto dovrebbe stare nella tabella `prodotti`.

Perché serve

La 2NF aiuta a ridurre duplicazioni e anomalie negli aggiornamenti.

Non è il primo concetto di normalizzazione da memorizzare a forza. **Prima capisci bene 1NF e le relazioni. Poi questo livello diventa molto più naturale.**

Un consiglio pratico

Se la 2NF ti sembra astratta, non sei tu il problema. All'inizio succede spesso.

L'importante è capire l'idea generale: i dati devono vivere dove dipendono davvero.

Normalizzazione 3NF

La terza forma normale e l'idea di evitare dipendenze indirette.

Qui elimini i passaggi indiretti che sporcano il modello

La terza forma normale, o 3NF, dice che le colonne non chiave devono dipendere dalla chiave primaria, non da altre colonne non chiave.

In pratica: evita passaggi indiretti che creano duplicazioni e confusione.

Pensa a una scheda cliente con `cap` e `citta`. Se la città dipende dal CAP, e il CAP dipende dal cliente, hai una dipendenza indiretta.

Un esempio intuitivo

Se in una tabella ordini salvi anche informazioni che dipendono dal cliente, e non dall'ordine, rischi di ripetere dati inutilmente.

Per esempio, se il nome della città dipende dal cliente, spesso non ha senso copiarlo dentro ogni ordine senza una buona ragione.

Un esempio in tabella

cliente_id	nome	cap	citta
1	Luca	00100	Roma
2	Marta	00100	Roma

Se `00100` indica sempre `Roma`, ripetere la città in ogni cliente può creare errori. Qualcuno potrebbe scrivere `Rma` in una riga e `Roma` in un'altra.

Una struttura più pulita può separare i CAP in una tabella dedicata.

Perché è utile

La 3NF riduce il rischio di incoerenze.

Se un dato vive nel posto giusto, quando cambia lo aggiorni una volta sola. **È questo uno dei vantaggi più grandi della normalizzazione.**

Un modo semplice per ricordarla

Se una colonna sembra dipendere più da un'altra colonna che dalla chiave della riga, fermati un attimo.

Potrebbe essere il segnale che quel dato sta nel posto sbagliato.

Denormalizzazione

Quando duplicare un po' di dati può migliorare letture o report.

A volte un po' di disordine controllato è una scelta intenzionale

La denormalizzazione è la scelta di duplicare alcuni dati per ottenere vantaggi pratici, spesso in velocità di lettura o semplicità di certe query.

È il contrario della normalizzazione pura, ma non è per forza sbagliata. Dipende dal problema che stai cercando di risolvere.

Pensa a una ricetta che copi su un foglio da tenere vicino ai fornelli. La ricetta originale resta nel libro, ma la copia ti fa risparmiare tempo mentre cucini.

Nel database, a volte copi un dato in più per leggere più in fretta.

Quando può avere senso

Può essere utile quando:

- hai report pesanti letti continuamente
- fai molte query complicate con join costose

- la velocità di lettura conta più della perfezione teorica

Qual è il prezzo da pagare

Il rischio è introdurre incoerenze.

Se un dato è copiato in più posti, quando cambia devi ricordarti di aggiornarlo ovunque. **La denormalizzazione va quindi fatta con un motivo preciso, non per comodità momentanea.**

Un esempio semplice

In una tabella `ordini` potresti salvare anche `nome_cliente`, oltre a `cliente_id`, per rendere un report più rapido da leggere.

```
SELECT numero_ordine, nome_cliente, totale
FROM ordini_report;
```

Questa scelta può essere utile per un report. Ma se il cliente cambia nome, devi sapere come aggiornare anche la copia.

:::tip Prima normalizza per capire bene i dati. Denormalizza solo quando hai un motivo concreto e misurabile. :::

Backup e restore

Copie di sicurezza e ripristino del database quando qualcosa va storto.

Il backup è la rete di sicurezza del database

Un backup è una copia di sicurezza. Il restore è il processo con cui torni a quella copia quando ne hai bisogno.

Una regola semplice ma importantissima: **un backup vale davvero solo se sai ripristinarlo.**

Pensa a una foto importante salvata solo sul telefono. Dire "la copierò prima o poi" non ti protegge. Una copia vera, conservata bene e verificata, sì.

Perché non basta dire "abbiamo i backup"

Molti problemi nascono non dall'assenza di backup, ma dal fatto che nessuno ha mai provato seriamente il restore.

Quando arriva il momento del bisogno, scoprire che il recupero non funziona è troppo tardi.

Cosa tenere in mente

Un buon approccio include:

- fare backup con regolarità
- sapere dove sono conservati
- proteggere anche i backup
- testare periodicamente il ripristino

Un controllo semplice

Per ragionare bene sui backup, chiediti:

- quanto spesso faccio una copia?
- quanti dati posso permettermi di perdere?
- quanto tempo posso restare fermo?
- chi sa davvero fare il restore?

:::caution I backup contengono dati veri. Vanno protetti come il database principale. :::

Best practice SQL

Abitudini sane quando scrivi schema, query e manutenzione del database.

Le buone pratiche sono piccole scelte che fanno risparmiare molto dopo

In SQL la qualità raramente nasce da un colpo di genio. Nasce più spesso da abitudini sane ripetute con costanza.

Pensa alla manutenzione di una bicicletta. Gonfiare le gomme, controllare i freni e pulire la catena sembrano gesti piccoli. Insieme evitano grossi problemi.

- usa nomi chiari

- evita `SELECT *` quando non serve
- aggiungi vincoli
- testa `UPDATE` e `DELETE` con una `SELECT` prima
- fai backup regolari

Alcune abitudini da adottare subito

Scrivi query leggibili:

```
SELECT nome, email
FROM clienti
WHERE attivo = TRUE;
```

Questa versione dice chiaramente quali colonne vuoi, da dove arrivano e quale filtro usi.

Il valore vero di queste abitudini

Ogni singola pratica può sembrare piccola. Insieme, però, rendono il database più leggibile, più stabile e meno fragile.

Le best practice non servono a sembrare esperti. Servono a farti trovare meno problemi domani.

Un buon modo per usarle

Non serve inseguirle come una checklist perfetta in ogni momento. Serve usarle come bussola mentre progetti, scrivi e mantieni query e schema.

:::caution La pratica più importante con `UPDATE` e `DELETE` è controllare prima il filtro. Sono comandi utili, ma possono fare molti danni se usati senza attenzione. :::

Anti-pattern comuni

Errori ricorrenti da evitare quando lavori con SQL e con la struttura del database.

Un anti-pattern è una scorciatoia che poi presenta il conto

Un anti-pattern è una soluzione che all'inizio sembra comoda, ma nel tempo genera confusione, bug o lentezza.

È come mettere tutto in un cassetto "varie". Oggi fai prima. Tra un mese non trovi più niente.

Esempi comuni:

- tabelle senza chiave primaria
- duplicazioni inutili
- colonne che mischiano troppi significati
- query lunghissime senza alias chiari

Un esempio facile da riconoscere

Una colonna chiamata `dati_extra` che contiene pezzi diversi di informazione può sembrare comoda.

Il problema arriva quando vuoi cercare, filtrare o controllare quei dati. Il database non sa più bene cosa rappresenta quella colonna.

Perché vale la pena riconoscerli

Se impari a vedere questi schemi presto, eviti di costruire problemi che diventeranno costosi solo più avanti.

Riconoscere un anti-pattern è già metà della correzione.

Una postura utile

Quando una soluzione sembra fin troppo comoda, vale la pena chiedersi se stai semplificando davvero oppure stai solo spostando il problema più avanti nel tempo.

:::tip Se una scelta rende difficile spiegare il database a voce, spesso merita una seconda occhiata. :::

Import/export dati

Scambiare dati con CSV, JSON e altri formati in entrata o in uscita.

Un database non vive da solo

Un database comunica spesso con altri sistemi. Per questo è normale importare dati dentro il database o esportarli verso file e strumenti esterni.

Per questo è comune importare o esportare dati in formati come:

- CSV
- JSON
- fogli di calcolo

Pensa al database come a un magazzino. A volte riceve scatole da fuori. A volte prepara pacchi da spedire ad altri.

Quando succede nella pratica

Per esempio:

- carichi un CSV ricevuto da un fornitore
- esporti dati per un report
- trasferisci informazioni tra sistemi diversi

La cosa da non sottovalutare

Importare ed esportare non significa solo "spostare file". Vuol dire anche controllare formati, codifiche, separatori e qualità dei dati.

Un esempio di attenzione pratica

Un file CSV può sembrare semplice, ma può nascondere dettagli importanti:

- quale carattere separa le colonne?
- le date sono nel formato giusto?
- i testi hanno accenti e caratteri speciali?
- ci sono righe vuote o valori mancanti?

:::tip Prima importa un piccolo campione. Se tutto torna, passa al file completo. :::

Sicurezza del database

Buone pratiche per proteggere dati, accessi e operazioni sensibili.

La sicurezza non è un argomento separato dal database: ne fa parte

La sicurezza del database non è un accessorio finale. Fa parte del lavoro quotidiano, proprio come scrivere query corrette o progettare tabelle sensate.

Pensa a un archivio con documenti personali. Non basta ordinarlo bene. Devi anche decidere chi può aprirlo, copiarlo o modificarlo.

Buone abitudini:

- dare solo i permessi necessari
- evitare account condivisi
- proteggere i dati sensibili
- usare credenziali robuste

Una mentalità utile

Non pensare solo a "come entro". Pensa anche a:

- chi può vedere cosa
- chi può modificare cosa
- come proteggere i backup
- come registrare accessi ed errori importanti

Un database custodisce informazioni. Proteggerle è parte del suo mestiere.

Un primo principio semplice

Il principio del minimo privilegio dice: dai a ogni utente solo i permessi che gli servono davvero.

Se un'app deve solo leggere dati pubblici, non dovrebbe poter cancellare tabelle.

:::caution Anche i backup vanno protetti. Spesso contengono gli stessi dati sensibili del database principale. :::

Versioning e migrazioni database

Tenere traccia dei cambiamenti dello schema nel tempo senza perdere il controllo.

Lo schema cambia, e quei cambiamenti vanno trattati con disciplina

Le migrazioni sono cambiamenti controllati dello schema. Aggiungere una colonna, creare una tabella, modificare un vincolo: tutto questo dovrebbe lasciare una traccia chiara.

Versionarle aiuta il team a sapere sempre quale struttura deve avere il database.

Pensa alle versioni di un regolamento condominiale. Se ognuno ha una copia diversa, nascono discussioni. Se ogni modifica ha data e ordine, tutti capiscono cosa vale.

Perché è così importante

Senza versioning, ogni ambiente rischia di diventare diverso dagli altri. Uno sviluppatore ha una colonna in più, un altro no, e iniziano confusione ed errori.

Le migrazioni portano ordine dove altrimenti regnerebbe memoria personale e improvvisazione.

Un esempio semplice

Una migrazione può dire:

```
ALTER TABLE clienti  
ADD telefono VARCHAR(30);
```

Questa modifica aggiunge la colonna `telefono`. Salvandola in una migrazione, puoi applicarla in modo ordinato anche su altri ambienti.

Una buona abitudine da adottare presto

Anche in progetti piccoli, tenere traccia dei cambiamenti dello schema è un investimento utile.

Ti evita di affidarti al ricordo e rende il database parte integrante del lavoro di squadra.

:::caution Cambiare o cancellare colonne con dati reali richiede prudenza. Prima pensa a backup, compatibilità e piano di ritorno. :::

ETL

Extract, Transform, Load spiegato come un flusso pratico di lavoro sui dati.

ETL è il viaggio dei dati da una sorgente a una destinazione

ETL significa:

1. **Extract:** prendi i dati
2. **Transform:** li sistemi
3. **Load:** li carichi nel database

Pensa a un pacco che arriva in magazzino. Prima lo scarichi, poi controlli e sistemi il contenuto, infine lo metti nello scaffale giusto.

Un esempio concreto

Immagina di ricevere ogni giorno un file CSV da un altro sistema.

Prima lo leggi, poi sistemi i valori sporchi o incompleti, infine carichi i dati nel database. **Questo è ETL.**

Cosa succede nella fase di trasformazione

Durante **Transform** potresti:

- correggere date scritte in formati diversi
- eliminare righe duplicate
- convertire testi in maiuscolo o minuscolo
- controllare valori mancanti

Perché è importante

È un concetto molto comune quando i dati arrivano da sorgenti diverse e non sono subito pronti per entrare nel tuo sistema.

Il valore vero dell'ETL

Non è solo spostare dati da A a B. È fare in modo che arrivino puliti, coerenti e nel formato giusto.

Questo fa una grande differenza quando i sistemi coinvolti diventano molti.

:::note ETL non è un singolo comando SQL. È un modo di organizzare un processo. :::

EXPLAIN e query plan

Capire come il database decide di eseguire una query e dove perde tempo.

Quando una query è lenta, non si indovina: si guarda il piano

EXPLAIN serve a vedere il piano di esecuzione di una query. In altre parole, ti mostra come il database pensa di arrivare al risultato.

È come chiedere a un navigatore: "che strada pensi di fare?" Prima di cambiare percorso, vuoi vedere il tragitto scelto.

```
EXPLAIN
SELECT *
FROM ordini
WHERE totale > 100;
```

Questa query non serve a mostrare gli ordini. Serve a mostrare come il database intende cercarli.

Perché è così utile

Guardare il query plan aiuta a capire se:

- il database sta leggendo troppe righe
- manca un indice utile
- una join è costosa
- la query può essere scritta meglio

Non ti dà una soluzione automatica, ma ti dà la mappa del problema.

Cosa guardare all'inizio

Quando sei alle prime armi, cerca soprattutto segnali semplici:

- il database legge tutta la tabella?
- usa un indice?
- stima tantissime righe?

:::tip Non serve capire ogni riga del piano subito. Inizia a usarlo come una lente per fare domande migliori. :::

JSON e dati semi-strutturati

Lavorare con campi JSON dentro un database relazionale senza perdere la bussola.

Non tutti i dati sono rigidi come una tabella classica

Alcuni dati hanno una parte stabile e una parte più flessibile. I campi JSON servono quando vuoi conservare informazioni con struttura meno rigida dentro un database relazionale.

SQL resta relazionale, ma molti database moderni sanno gestire anche dati semi-strutturati.

Pensa a una scheda prodotto. Nome, prezzo e codice sono campi stabili. Le caratteristiche tecniche, invece, possono cambiare molto da prodotto a prodotto.

Quando può essere utile

Per esempio:

- impostazioni personalizzate di un utente
- metadati variabili
- configurazioni che cambiano da caso a caso

Il punto da ricordare

JSON è utile, ma non dovrebbe diventare una scusa per buttare tutto in un campo indistinto. **Se un dato è davvero strutturato e importante, spesso merita ancora una colonna o una tabella dedicata.**

Un esempio piccolo

Un campo `preferenze` potrebbe contenere dati come:

```
{  
  "tema": "scuro",  
  "lingua": "it"  
}
```

Queste preferenze possono cambiare nel tempo senza obbligarti subito a modificare molte colonne.

La scelta giusta è quasi sempre mista

Molti progetti usano una base relazionale solida e aggiungono JSON solo dove la flessibilità serve davvero.

Questo equilibrio di solito è più sano di un approccio tutto o niente.

Gestione di grandi dataset

Idee pratiche quando i dati diventano molto numerosi e le abitudini piccole non bastano più.

Quando i dati crescono, cambiano anche le domande giuste

Con pochi dati, quasi tutto sembra veloce. Quando i dati crescono davvero, iniziano a contare scelte che prima sembravano secondarie.

È come cucinare per quattro persone o per quattrocento. La ricetta può essere simile, ma organizzazione, tempi e strumenti cambiano molto.

Diventano importanti:

- indici ben scelti
- query efficienti
- partizionamento
- monitoraggio

- archiviazione dei dati vecchi

La lezione principale

Gestire grandi dataset non significa solo avere hardware migliore. Significa soprattutto progettare meglio query, struttura e processi.

La scala grande punisce le scorciatoie piccole.

Un esempio pratico

Una query che legge tutti gli ordini può andare bene con 500 righe.

Con 50 milioni di righe, la stessa query può diventare un problema. A quel punto servono filtri, indici, archiviazione e magari partizionamento.

Cosa cambia nella mentalità

Con molti dati diventa ancora più importante pensare in anticipo a manutenzione, osservabilità e crescita futura.

Non basta che qualcosa funzioni oggi. Deve continuare a funzionare bene anche quando il volume aumenta molto.

:::tip Quando progetti una tabella, chiediti anche come verrà letta tra un anno, non solo come viene scritta oggi. :::

Logging e monitoring

Tenere sotto controllo salute, errori e carico del database nel tempo.

Se non osservi il database, lavori quasi al buio

Logging e monitoring servono a capire cosa sta succedendo davvero. Quando il sistema rallenta, fallisce o si comporta in modo strano, hai bisogno di tracce affidabili.

- **logging:** registra eventi ed errori
- **monitoring:** osserva salute, tempi e carico

Perché sono diversi

Il logging racconta episodi specifici. Il monitoring ti mostra l'andamento generale.

Uno è più vicino al diario degli eventi. L'altro è più simile al cruscotto di un'auto.

Cosa potresti osservare

In un database può essere utile controllare:

- query lente
- errori frequenti
- spazio su disco
- numero di connessioni
- tempi medi di risposta

Perché sono indispensabili

Senza visibilità, i problemi arrivano prima di essere capiti. **Con buoni segnali, puoi intervenire prima che un piccolo rallentamento diventi un problema serio.**

La loro utilità cresce con il sistema

Più il progetto cresce, più diventa importante poter ricostruire cosa è successo e quando.

Questo vale sia per la diagnosi tecnica sia per la serenità operativa del team.

:::tip Non aspettare il primo problema grave per attivare logging e monitoring. Prepararli prima è molto più tranquillo. :::

Introduzione a ORM

Object Relational Mapping spiegato senza gergo inutile.

Un ORM è un ponte tra il tuo codice e il database

ORM significa **Object Relational Mapping**. In pratica è uno strumento che ti permette di lavorare con il database usando oggetti o classi del tuo linguaggio, senza scrivere sempre SQL a mano.

Pensa a un cameriere che prende il tuo ordine e lo porta in cucina. Tu non parli direttamente con ogni cuoco, ma la cucina esiste comunque e deve lavorare bene.

Un ORM fa qualcosa di simile: il tuo codice parla con l'ORM, e l'ORM prepara le query per il database.

Un esempio mentale

Invece di scrivere a mano:

```
SELECT nome, email
FROM clienti
WHERE id = 1;
```

con un ORM potresti usare un metodo del tuo linguaggio, per esempio "trova il cliente con id 1".

Il risultato sembra più comodo, ma sotto il database riceve comunque una query.

Perché non sostituisce SQL

Un ORM può rendere comode molte operazioni ripetitive. Ma sotto c'è comunque un database relazionale, con query, join, filtri e performance da capire.

Sapere SQL resta fondamentale, anche quando usi un ORM. In molti casi è proprio ciò che ti permette di capire quando l'ORM ti sta aiutando e quando invece ti sta nascondendo un problema.

Il modo migliore di vederlo

Un ORM è un assistente, non un sostituto della comprensione.

Se sai già cosa significa fare una join, filtrare con `WHERE` o raggruppare con `GROUP BY`, allora usi l'ORM con molta più consapevolezza.

:::tip Quando qualcosa è lento, prova sempre a guardare la query SQL generata dall'ORM. Spesso il problema diventa molto più chiaro. :::

Analisi delle performance

Come ragionare sulle query lente e sui colli di bottiglia senza andare a tentoni.

Migliorare le performance significa osservare, non indovinare

Quando qualcosa è lento, il primo riflesso giusto è misurare. Non basta dire "mi sembra lenta". Bisogna capire dove si perde tempo.

È come sentire un rumore strano in auto. Prima di cambiare pezzi a caso, apri il cofano, ascolti, controlli e cerchi la causa.

Controlla soprattutto:

- query lente
- tabelle grandi
- indici mancanti
- join costose
- uso inutile di `SELECT *`

La domanda giusta

Non chiederti solo: "come faccio a renderla veloce?"

Chiediti prima: **"cosa sta rallentando davvero?"**

È questo il passo che separa una correzione utile da un tentativo casuale.

Un piccolo percorso pratico

Quando una query è lenta, puoi procedere così:

1. guarda quanto tempo impiega
2. controlla quante righe legge
3. osserva il piano con `EXPLAIN`
4. verifica se manca un indice utile
5. riscrivi la query solo dopo aver capito il problema

:::caution Una query veloce con 100 righe può diventare lenta con 10 milioni di righe. I test devono assomigliare il più possibile alla realtà. :::

Un'abitudine molto utile

Guarda il problema nel contesto: volume dei dati, frequenza della query, presenza di indici, piano di esecuzione e pattern di utilizzo.

Le performance non si migliorano bene guardando un solo pezzo isolato.

Sharding

Il concetto generale di dividere i dati tra server diversi.

Qui non dividi solo una tabella: dividi il carico su più macchine

Lo sharding distribuisce i dati su più server. Invece di tenere tutto in una sola macchina, dividi il carico in più parti.

Pensa a una catena di negozi. Se un solo negozio non riesce a servire tutti i clienti, apri più sedi. Ogni sede gestisce una parte del lavoro.

Perché si fa

Quando un sistema cresce moltissimo, un solo server può non bastare più. In questi casi si può distribuire il carico e i dati su più nodi.

Per esempio, potresti mettere alcuni clienti su un server e altri clienti su un altro server, seguendo una regola precisa.

Il prezzo del vantaggio

Aiuta a scalare sistemi molto grandi, ma aumenta parecchio la complessità tecnica e operativa.

Per questo è un concetto da conoscere, ma non una soluzione da applicare troppo presto.

Con lo sharding diventano più difficili domande come:

- dove si trova questo dato?
- come faccio una query che attraversa più server?
- come faccio backup e manutenzione?
- cosa succede se uno shard ha molto più traffico degli altri?

La lezione pratica

Prima di pensare allo sharding, di solito conviene migliorare schema, indici, query e infrastruttura normale.

Molti progetti non ne avranno mai bisogno, ed è perfettamente normale.

Differenze tra dialetti SQL

MySQL, PostgreSQL, SQLite e SQL Server a confronto in modo semplice.

SQL è una lingua comune con accenti diversi

SQL è uno standard, ma ogni database ha il suo accento. Le idee di base sono molto simili, però alcuni dettagli cambiano.

Pensa all'italiano parlato in regioni diverse. La lingua è la stessa, ma certe parole, espressioni e abitudini cambiano.

Controlla sempre:

- tipi di dati
- auto-increment
- funzioni disponibili
- sintassi di `LIMIT` o equivalenti
- supporto a feature avanzate

Perché conta per chi studia

All'inizio può sembrare fastidioso. In realtà è normale. **Prima impari la logica comune, poi impari le differenze del database che stai usando.**

Questo approccio evita di confondere il concetto con la variante.

Un esempio semplice

Per limitare i risultati, molti database usano:

```
SELECT nome
FROM clienti
LIMIT 10;
```

SQL Server usa spesso una forma diversa, come `TOP`.

Non cambia l'idea: vuoi solo vedere pochi risultati.

La buona notizia

Una volta capiti bene i concetti centrali, passare da un dialetto all'altro è molto più facile di quanto sembri.

Di solito cambi alcuni dettagli, non l'intero modo di ragionare.

SQL vs NoSQL

Quando ha senso usare SQL e quando può avere senso un approccio NoSQL.

Non sono due squadre rivali: sono strumenti diversi

SQL e NoSQL sono strumenti diversi, non squadre rivali. La scelta dipende da come sono fatti i dati e da cosa devi farci.

Pensa a due tipi di contenitori. Una cassettera è perfetta per oggetti ordinati per categoria. Uno scatolone flessibile è più comodo quando gli oggetti cambiano forma spesso.

SQL è spesso ottimo quando:

- i dati hanno struttura chiara
- servono relazioni forti
- le transazioni contano molto

NoSQL può essere utile quando la struttura è più flessibile o il modello di accesso è molto particolare.

Un esempio concreto

Un gestionale ordini ha spesso bisogno di tabelle collegate, vincoli e transazioni affidabili. Qui SQL è una scelta molto naturale.

Un sistema che salva eventi molto vari, con campi che cambiano spesso, potrebbe invece valutare un database NoSQL.

Il punto importante per chi inizia

Capire bene SQL ti aiuta anche a capire meglio NoSQL. Se conosci bene tabelle, relazioni e vincoli, capisci più facilmente cosa stai scegliendo di tenere e cosa stai scegliendo di sacrificare in altri modelli.

La domanda migliore da farsi

Invece di chiederti "qual è il migliore in assoluto?", chiediti:

"quale modello si adatta meglio ai dati e ai problemi che ho davanti?"

:::note In molti progetti reali convivono più strumenti. Non devi trasformare questa scelta in una gara. :::

Partizionamento delle tabelle

Dividere una tabella grande in parti più piccole per gestirla meglio.

Quando una tabella diventa troppo grande, dividerla può aiutare

Il partizionamento divide una tabella molto grande in sezioni più piccole. Per esempio puoi separare i dati per mese, per anno o per area geografica.

Questo può aiutare su dataset grandi e query mirate.

È come dividere le bollette in raccoglitori per anno. Se cerchi una bolletta del 2025, non apri tutti i raccoglitori della casa.

Perché funziona

Se la query cerca solo un pezzo del totale, il database può evitare di attraversare tutto.

Non è una bacchetta magica, ma in certi scenari è uno strumento molto utile.

Un esempio mentale

Una tabella `ordini` può essere divisa per anno:

- `ordini_2024`
- `ordini_2025`
- `ordini_2026`

Dal punto di vista dell'utente puoi continuare a pensare a una tabella unica, ma il database sa che i dati sono organizzati in parti.

Quando non basta da solo

Il partizionamento non sostituisce query scritte bene, indici sensati e una buona progettazione.

Aiuta molto, ma funziona davvero bene quando si combina con altre scelte intelligenti.

:::caution Il partizionamento aggiunge complessità. Ha senso quando il volume dei dati è davvero grande o quando hai un motivo operativo chiaro. :::

Testing di query SQL

Verificare che le query facciano davvero ciò che ti aspetti, non solo ciò che speri.

Anche le query meritano test, non solo il codice dell'applicazione

Una query può essere sintatticamente corretta e comunque fare la cosa sbagliata. Per questo anche le query vanno testate.

È come una calcolatrice: può darti un numero, ma devi comunque sapere se hai scritto l'operazione giusta.

È utile avere:

- dati di prova
- risultati attesi
- controlli automatici dove possibile

Cosa stai verificando davvero

Quando testi una query, stai controllando:

- che selezioni le righe giuste
- che escluda quelle sbagliate
- che faccia i calcoli corretti
- che continui a funzionare quando cambiano i dati di contesto

Testare significa ridurre sorprese, non perdere tempo.

Un esempio piccolo

Se una query deve trovare solo clienti attivi, prepara dati di prova con:

- un cliente attivo
- un cliente non attivo
- un cliente con dati mancanti

Poi controlla che nel risultato compaia solo chi deve comparire.

:::tip I casi strani sono preziosi. Spesso sono proprio loro a scoprire errori nascosti. :::

Gestione utenti e permessi

Controllare chi può leggere, scrivere o amministrare il database.

Un database serio non è una stanza aperta a tutti

Gestire utenti e permessi significa decidere chi può fare cosa. Non tutte le persone o applicazioni che usano un database devono avere gli stessi poteri.

Pensa alle chiavi di un edificio. Non dai la chiave della cassaforte a chi deve solo entrare nella sala riunioni.

Di solito si gestiscono:

- utenti
- ruoli
- permessi di lettura
- permessi di scrittura
- permessi amministrativi

Perché conta così tanto

Se dai troppi permessi a tutti, aumenti il rischio di:

- errori accidentali
- modifiche indesiderate
- accessi impropri

La regola più sana è dare a ciascuno solo ciò che serve davvero.

Un esempio semplice

Un'applicazione che mostra un catalogo prodotti potrebbe avere solo il permesso di leggere.

Un pannello amministrativo, invece, potrebbe avere anche il permesso di modificare prezzi e descrizioni.

:::note Dare meno permessi non significa fidarsi meno delle persone. Significa ridurre il danno possibile se qualcosa va storto. :::

Stored procedure avanzate

Controllo di flusso, condizioni, cicli e variabili dentro le stored procedure.

Qui il database smette di essere solo una lista di query isolate

Dentro una stored procedure puoi trovare logica più ricca di una semplice sequenza lineare. Per questo si parla di versioni più avanzate.

Pensa a una ricetta con scelte e ripetizioni: se l'acqua bolle, aggiungi la pasta; se non bolle, aspetta. La procedura può fare qualcosa di simile con i dati.

Per esempio puoi incontrare:

- IF
- CASE
- cicli
- variabili locali

Cosa significa in pratica

Vuol dire che il database può prendere piccole decisioni da solo, ripetere un passaggio o scegliere un comportamento in base ai dati.

Questo può essere utile, ma va usato con equilibrio. **Troppa logica nascosta nel database può rendere il progetto più difficile da seguire.**

Un esempio mentale

Una procedura potrebbe controllare il totale di un ordine:

- se il totale supera una soglia, applica uno sconto
- altrimenti lascia il prezzo normale

Questa è logica, non solo lettura o scrittura di dati.

Il criterio più sano

Usa questa potenza quando porta chiarezza o affidabilità reali.

Se invece complica troppo la comprensione del sistema, probabilmente stai spostando troppa logica nel posto sbagliato.

:::tip Se una procedura avanzata diventa lunga da spiegare, forse merita di essere divisa o documentata meglio. :::

Cursori

Scorrere i risultati una riga alla volta quando un'elaborazione insieme non basta.

SQL ama lavorare per insiemi, ma non sempre basta

Un cursore permette di scorrere un risultato riga per riga. È come passare il dito lungo un elenco, fermandoti ogni volta su un elemento.

Di solito SQL preferisce lavorare su tante righe insieme. Il cursore è l'eccezione: prende una riga, la guarda, poi passa alla successiva.

Quando entra in scena

I cursori diventano utili in casi particolari, quando davvero hai bisogno di trattare i risultati uno alla volta.

Detto questo, SQL dà il meglio di sé quando ragiona su gruppi di righe insieme. **Per questo i cursori esistono, ma non sono lo strumento da preferire per tutto.**

Un esempio mentale

Se devi calcolare lo stesso aggiornamento per tutti i prodotti, una normale query `UPDATE` è spesso la scelta migliore.

Se invece devi chiamare una procedura diversa per ogni riga, con passaggi che dipendono dal risultato precedente, un cursore può avere senso.

La regola pratica

Se riesci a risolvere il problema con una query normale, in genere è meglio. Se il problema richiede una sequenza riga per riga, allora il cursore può avere senso.

Perché si consiglia prudenza

I cursori possono essere più lenti e più complessi da gestire rispetto alle operazioni insiemistiche tipiche di SQL.

Conoscerli è utile. Usarli come prima scelta quasi mai.

:::tip Prima di scegliere un cursore, prova a chiederti: "posso risolvere questo problema lavorando su tutte le righe insieme?" :::

Stored procedure

Blocchi di istruzioni salvati direttamente nel database per riutilizzare logica SQL.

Una stored procedure è una ricetta già pronta

Una stored procedure è un blocco di istruzioni salvato nel database. Invece di riscrivere sempre la stessa sequenza di comandi, puoi darle un nome e riutilizzarla.

È come salvare una ricetta con un titolo. Invece di riscrivere ogni volta tutti i passaggi, dici "prepara questa ricetta".

Un esempio piccolo

```
CREATE PROCEDURE aggiorna_prezzi ()
BEGIN
    UPDATE prodotti
    SET prezzo = prezzo * 1.10;
END;
```

Perché può essere utile

Una stored procedure è comoda quando:

- una sequenza di passaggi si ripete spesso
- vuoi centralizzare una logica vicino ai dati
- vuoi richiamare un'operazione complessa con un nome semplice

Un dettaglio importante

La sintassi cambia tra database diversi. **Il concetto però resta stabile:** salvi una procedura e poi la richiami quando ti serve.

:::caution Una procedura può modificare molti dati. Prima di usarla in un database reale, controlla bene cosa fa e su quali righe lavora. :::

Trigger

Azioni automatiche che scattano quando una tabella cambia.

Un trigger è una reazione automatica

Un trigger è una regola automatica. Succede un evento su una tabella, e il database reagisce.

Può scattare prima o dopo un `INSERT` , `UPDATE` o `DELETE` .

Un'immagine utile

Pensa a una porta con un sensore: appena qualcuno passa, succede qualcosa. Il trigger lavora in modo simile, ma sugli eventi del database.

Quando è utile

Può servire per:

- registrare modifiche in una tabella di log
- controllare certe regole automaticamente
- aggiornare dati collegati

Un esempio mentale

Immagina una tabella `prodotti` e una tabella `log_modifiche`. Ogni volta che cambia il prezzo di un prodotto, un trigger potrebbe registrare chi ha fatto la modifica e quando.

Così non devi ricordarti di scrivere la riga di log in ogni punto dell'applicazione.

Il lato da gestire con attenzione

I trigger possono essere potenti, ma anche un po' invisibili. Se un comportamento importante dipende da un trigger, chi legge il codice deve saperlo.

Per questo vanno usati con chiarezza e moderazione.

:::caution Se un trigger modifica altri dati senza che sia evidente, il sistema può diventare difficile da capire e da testare. :::

Funzioni definite dall'utente

Funzioni SQL create da te per restituire valori o risultati riutilizzabili.

Quando vuoi insegnare al database una piccola operazione nuova

Una funzione definita dall'utente permette di aggiungere una nuova funzione al tuo database. È simile a una stored procedure, ma in genere nasce per restituire un valore o un risultato.

Pensa a una calcolatrice con un tasto personalizzato. Premi quel tasto, gli dai un numero, e lei ti restituisce il risultato del calcolo che usi spesso.

Un esempio piccolo

```
CREATE FUNCTION prezzo_con_iva(prezzo DECIMAL(10,2))
RETURNS DECIMAL(10,2)
BEGIN
    RETURN prezzo * 1.22;
END;
```

Perché può tornare utile

Se hai un calcolo o una trasformazione che si ripete spesso, racchiuderla in una funzione può rendere il codice più ordinato.

Invece di riscrivere sempre la stessa formula, dai un nome a quell'idea.

Potresti poi usarla in una query:

```
SELECT nome, prezzo_con_iva(prezzo) AS prezzo_finale
FROM prodotti;
```

Qui `prezzo_con_iva` restituisce un valore calcolato per ogni prodotto.

Una distinzione semplice da ricordare

Non è una regola assoluta per tutti i database, ma puoi pensare così:

- la **procedura** esegue passi
- la **funzione** restituisce qualcosa

Questa scorciatoia mentale aiuta molto quando inizi.

:::note La sintassi delle funzioni cambia parecchio tra PostgreSQL, MySQL, SQL Server e altri database. L'idea di base, però, resta la stessa. :::

Alias

Come usare AS per dare soprannomi temporanei a colonne e tabelle.

A volte un nome più corto o più chiaro aiuta tantissimo

Un alias è un soprannome temporaneo. Lo usi per rendere i risultati più leggibili o per abbreviare i nomi nelle query più lunghe.

```
SELECT nome AS cliente  
FROM clienti;
```

Qui la colonna `nome` apparirà con il nome `cliente` nel risultato.

Perché sono comodi

Gli alias aiutano in due modi:

- rendono il risultato più parlante
- accorciano la scrittura quando ci sono tabelle con nomi lunghi

Alias sulle tabelle

Gli alias non valgono solo per le colonne. Puoi usarli anche sulle tabelle:

```
SELECT c.nome  
FROM clienti AS c;
```

Qui `c` è un soprannome della tabella `clienti`.

Dove diventano davvero preziosi

Nelle query con `JOIN`, gli alias migliorano tantissimo la leggibilità. Invece di ripetere sempre il nome completo della tabella, puoi usare un nome corto e chiaro.

Non cambiano i dati. Cambiano solo il modo in cui la query viene scritta o mostrata.

Operatori di confronto

Uguale, maggiore, minore, BETWEEN, LIKE e IN spiegati con esempi chiari.

Sono i simboli con cui SQL confronta i valori

Gli operatori di confronto dicono al database come confrontare i valori. Senza di loro, SQL non saprebbe se una riga è abbastanza grande, abbastanza piccola, uguale a ciò che stai cercando o compresa in un elenco.

Quelli che incontrerai più spesso

- `=` uguale
- `>` maggiore
- `<` minore
- `BETWEEN` dentro un intervallo
- `LIKE` testo che somiglia a un modello
- `IN` valore dentro una lista

Un esempio con un intervallo

```
SELECT *  
FROM ordini  
WHERE totale BETWEEN 50 AND 100;
```

Questa query prende gli ordini il cui totale è compreso tra 50 e 100.

LIKE : utile per il testo

Quando lavori con stringhe, **LIKE** ti aiuta a cercare un modello.

```
SELECT *  
FROM clienti  
WHERE nome LIKE 'L%';
```

Qui stai cercando nomi che iniziano con la lettera **L**.

IN : utile quando hai un piccolo elenco

Se vuoi confrontare un valore con più possibilità, **IN** rende la query più pulita:

```
SELECT *  
FROM clienti  
WHERE citta IN ('Roma', 'Milano', 'Torino');
```

Invece di scrivere tante condizioni con **OR**, metti tutto in una lista leggibile.

Il modo giusto di pensarli

Gli operatori di confronto sono il vocabolario di base dei filtri. **Più li conosci bene, più costruire condizioni diventa naturale.**

LIMIT e OFFSET

Come mostrare solo una parte dei risultati quando l'elenco è troppo lungo.

Qui impari a dosare il risultato

A volte non vuoi vedere tutte le righe. Vuoi solo le prime 5, oppure vuoi saltare le prime 10 e leggere le successive.

`LIMIT` e `OFFSET` servono proprio a questo.

```
SELECT *  
FROM clienti  
LIMIT 5;
```

Questa query restituisce solo le prime 5 righe del risultato.

Saltare le prime righe

Per saltare le prime righe:

```
SELECT *  
FROM clienti  
LIMIT 5 OFFSET 5;
```

Qui stai chiedendo: **"salta le prime 5 righe e poi mostrami le successive 5"**.

Perché è utile nella pratica

Questi strumenti sono molto usati quando costruisci:

- pagine di risultati
- liste prodotti
- tabelle amministrative
- schermate con paginazione

Un'accoppiata frequente

Spesso `LIMIT` e `OFFSET` si usano insieme a `ORDER BY`.

Per esempio, se vuoi mostrare i 10 ordini più recenti, prima ordini per data e poi limiti il risultato. **Senza ordine, il taglio delle righe può diventare poco prevedibile.**

Operatori logici

AND, OR e NOT per combinare più condizioni nella stessa query.

Quando una sola condizione non basta

Gli operatori logici servono a combinare più condizioni. Sono il modo con cui SQL ragiona su frasi come:

- clienti di Roma **e** attivi
- clienti di Roma **o** di Milano
- ordini **non** ancora spediti

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' AND attivo = TRUE;
```

AND richiede che entrambe le condizioni siano vere.

OR ne accetta almeno una. **NOT** la ribalta.

AND : tutto deve essere vero

AND funziona come una doppia porta. La riga passa solo se supera entrambe.

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' AND attivo = TRUE;
```

Qui restano solo i clienti che vivono a Roma **e** sono attivi.

OR : basta una delle due

Con **OR** la logica è più permissiva.

```
SELECT *  
FROM clienti  
WHERE citta = 'Roma' OR citta = 'Milano';
```

Qui basta che la città sia Roma oppure Milano.

NOT : capovolgere la condizione

NOT serve a negare una condizione.

```
SELECT *  
FROM clienti  
WHERE NOT attivo;
```

Questa query prende i clienti che non risultano attivi.

Perché conviene leggere la query come una frase

Quando una query ha più condizioni, il trucco migliore è leggerla ad alta voce in italiano semplice. Se la frase suona chiara, di solito anche la query è costruita bene.

Valori NULL

Come rappresentare e controllare l'assenza di un valore in SQL.

Quando un dato manca

NULL significa che un valore **non c'è** oppure **non è conosciuto**.

Non significa zero. Non significa testo vuoto. Non significa "no". Significa proprio: "qui il dato manca".

È come una casella lasciata vuota in un modulo. La casella esiste, ma nessuno ci ha scritto dentro nulla.

Cercare valori mancanti

Per trovare i clienti senza email, scrivi:

```
SELECT nome, email
FROM clienti
WHERE email IS NULL;
```

La parte importante è **IS NULL**.

Con **NULL** non si usa **=**, perché **NULL** non si comporta come un valore normale.

Questa query sembra naturale, ma è sbagliata:

```
SELECT nome, email
FROM clienti
WHERE email = NULL;
```

La forma corretta è:

```
SELECT nome, email
FROM clienti
WHERE email IS NULL;
```

In pratica non chiedi "email è uguale a mancante?". Chiedi "email è mancante?".

Cercare valori presenti

Il contrario di **IS NULL** è **IS NOT NULL**.

```
SELECT nome, email
FROM clienti
WHERE email IS NOT NULL;
```

Questa query mostra solo i clienti che hanno un'email salvata.

NULL , zero e testo vuoto

Questi tre casi sono diversi:

Valore	Significato
NULL	il dato manca
0	il dato c'è ed è zero
' '	il dato c'è ma è vuoto

Per esempio, se `sconto` è `NULL` , forse non sai ancora lo sconto. Se `sconto` è `0` , sai che lo sconto è zero.

Questa differenza è importante nei calcoli, nei filtri e nei report.

Quando lo incontrerai

`NULL` compare spesso quando:

- una colonna è facoltativa
- un dato non è stato ancora inserito
- un'informazione non è disponibile
- una `LEFT JOIN` non trova una riga collegata

Pensarlo come "dato mancante" è la scorciatoia mentale più utile.

Riepilogo rapido

- `NULL` significa dato mancante o sconosciuto.
- Non è uguale a zero.
- Non è uguale a testo vuoto.
- Si controlla con `IS NULL` .
- Il contrario si controlla con `IS NOT NULL` .

ORDER BY

Come ordinare i risultati in modo crescente o decrescente.

Dopo aver trovato i dati, puoi metterli in ordine

ORDER BY serve a scegliere l'ordine con cui il database ti mostra i risultati. Senza di lui, l'ordine non è qualcosa su cui conviene fare affidamento.

Pensa a una lista di nomi o a un elenco di prezzi. Stesso contenuto, ma un ordine diverso può renderlo molto più utile.

```
SELECT nome
FROM clienti
ORDER BY nome ASC;
```

ASC vuol dire crescente. **DESC** vuol dire decrescente.

Quando lo usi davvero

ORDER BY è utilissimo quando vuoi:

- nomi in ordine alfabetico
- prezzi dal più alto al più basso
- date dalla più recente alla più vecchia
- punteggi in classifica

Un altro esempio chiaro

```
SELECT nome, totale
FROM ordini
ORDER BY totale DESC;
```

Qui gli ordini con il totale più alto compariranno per primi.

Un dettaglio importante

L'ordinamento avviene sul risultato della query. Prima selezioni le righe, poi decidi come disporle.

Questo rende **ORDER BY** perfetto da usare dopo **WHERE**, quando hai già filtrato ciò che ti interessa.

SELECT

Come leggere i dati da una tabella scegliendo colonne e risultati utili.

Leggere dati da una tabella

SELECT è il comando che usi quando vuoi **vedere dei dati**.

Non aggiunge nulla, non cancella nulla e non modifica le informazioni salvate. Fa una cosa sola: chiede al database di mostrarti qualcosa.

Immagina una tabella `clienti` fatta così:

id	nome	email
1	Luca	luca@example.com
2	Marta	marta@example.com

Se vuoi vedere tutto quello che contiene, puoi scrivere:

```
SELECT *  
FROM clienti;
```

Il simbolo `*` significa "tutte le colonne". Quindi questa query dice: "mostrami tutto dalla tabella `clienti`".

È comoda quando stai esplorando una tabella per la prima volta. È un po' come aprire un cassetto per vedere cosa c'è dentro.

Chiedere solo quello che ti serve

Nella pratica, però, spesso non ti serve tutto. Magari vuoi vedere solo il nome e l'email dei clienti:

```
SELECT nome, email  
FROM clienti;
```

Il risultato diventa più pulito:

nome	email
Luca	luca@example.com
Marta	marta@example.com

Hai ancora gli stessi clienti, ma stai guardando solo le colonne utili per la tua domanda.

Questa è una buona abitudine: più la query è precisa, più è facile da leggere.

Come leggere una query **SELECT**

Una query di lettura molto semplice ha due parti:

- **SELECT** dice **quali colonne** vuoi vedere
- **FROM** dice **da quale tabella** prenderle

Questa query:

```
SELECT nome, email  
FROM clienti;
```

si può leggere così:

*"Vai nella tabella **clienti** e mostrami le colonne **nome** ed **email**."*

Quando sei agli inizi, tradurre la query in italiano aiuta molto. Se la frase italiana suona confusa, probabilmente anche la query merita di essere riscritta meglio.

Perché non usare sempre **SELECT ***

SELECT * va benissimo per curiosare in una tabella piccola o per fare prove veloci.

Quando però scrivi una query che vuoi conservare, condividere o usare in un'applicazione, è meglio indicare le colonne per nome:

```
SELECT nome, email  
FROM clienti;
```

Così chi legge capisce subito quali dati ti interessano. Inoltre eviti di portarti dietro colonne inutili, magari pesanti o sensibili.

Un esempio con i prodotti

Se hai una tabella `prodotti` con colonne `id`, `nome`, `prezzo` e `disponibile`, e vuoi vedere solo nome e prezzo, scrivi:

```
SELECT nome, prezzo  
FROM prodotti;
```

Il gesto è sempre lo stesso: scegli la tabella, poi scegli le colonne.

Riepilogo rapido

- `SELECT` legge dati.
- `FROM` indica la tabella.
- `*` significa "tutte le colonne".
- Nelle query curate, è meglio indicare solo le colonne che servono.

WHERE

Come filtrare le righe che ti servono davvero, senza leggere tutto.

Scegliere quali righe vedere

`WHERE` serve a dire al database: **mostrami solo le righe che rispettano questa condizione.**

Senza `WHERE`, una query legge tutte le righe della tabella. Con `WHERE`, invece, restringi il risultato.

È come cercare in una rubrica solo le persone che vivono a Roma, invece di leggere tutti i contatti uno per uno.

```
SELECT nome, citta
FROM clienti
WHERE citta = 'Roma';
```

Questa query significa:

*“Mostrami nome e città dei clienti che hanno **Roma** nella colonna **citta** .”*

Se nella tabella ci sono clienti di città diverse, vedrai solo quelli di Roma.

La condizione dopo **WHERE**

La parte importante è questa:

```
WHERE citta = 'Roma'
```

SQL controlla la condizione una riga alla volta.

Se la riga ha **Roma** nella colonna **citta** , entra nel risultato. Se ha **Milano** , **Napoli** o qualsiasi altro valore, viene esclusa.

Nota: I testi si scrivono di solito tra apici singoli, come **'Roma'** . I numeri invece si scrivono senza apici, come **18** .

Un esempio con i numeri

Ora immaginiamo una tabella **ordini** :

id	totale
1	35.00
2	120.00
3	89.90

Per vedere solo gli ordini sopra 100, scrivi:

```
SELECT id, totale
FROM ordini
WHERE totale > 100;
```

Il risultato sarà:

id	totale
2	120.00

La query non ha “saltato” le altre righe per caso. Le ha controllate e ha tenuto solo quella in cui `totale > 100` era vero.

Gli operatori più comuni

Dopo `WHERE` puoi usare diversi operatori:

Operatore	Significato	Esempio
<code>=</code>	uguale a	<code>citta = 'Roma'</code>
<code><></code>	diverso da	<code>stato <> 'annullato'</code>
<code>></code>	maggiore di	<code>totale > 100</code>
<code><</code>	minore di	<code>eta < 18</code>
<code>>=</code>	almeno	<code>prezzo >= 10</code>
<code><=</code>	al massimo	<code>quantita <= 5</code>

Non serve impararli tutti a memoria subito. Li userai così spesso che diventeranno familiari.

Testi, numeri e date

Il valore che confronti deve essere scritto nel modo giusto.

Per un testo usi gli apici:

```
SELECT nome
FROM clienti
WHERE citta = 'Roma';
```

Per un numero no:

```
SELECT id, totale
FROM ordini
WHERE totale >= 50;
```

Per una data si usa spesso il formato `AAAA-MM-GG` :

```
SELECT id, data_ordine
FROM ordini
WHERE data_ordine >= '2026-01-01';
```

Il database capisce che quel testo rappresenta una data se la colonna `data_ordine` è stata creata con un tipo adatto.

Una distinzione importante

Con `SELECT` , `WHERE` filtra solo quello che vedi. Non cambia i dati.

Ma più avanti userai `WHERE` anche con comandi come `UPDATE` e `DELETE` . Lì diventa molto più delicato, perché decide quali righe modificare o cancellare.

Attenzione: Prima di modificare o cancellare dati, controlla sempre il filtro con una `SELECT` . È una piccola abitudine che evita grandi guai.

Riepilogo rapido

- `WHERE` filtra le righe.
 - La condizione può essere vera o falsa.
 - SQL controlla la condizione riga per riga.
 - Con `UPDATE` e `DELETE`, un `WHERE` sbagliato può toccare troppe righe.
-

Vincoli

PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK e DEFAULT spiegati in modo pratico.

Regole che proteggono i dati

I **vincoli** sono regole che il database controlla prima di salvare un dato.

Non servono a complicare la vita. Servono a evitare dati rotti, duplicati o incoerenti.

È come avere qualcuno all'ingresso di un magazzino che controlla: "questo pacco ha l'etichetta?", "questo codice esiste già?", "questo valore ha senso?".

I vincoli più comuni

Ecco quelli che incontrerai più spesso:

Vincolo	A cosa serve
<code>PRIMARY KEY</code>	identifica ogni riga
<code>FOREIGN KEY</code>	collega una tabella a un'altra
<code>UNIQUE</code>	impedisce valori duplicati
<code>NOT NULL</code>	rende una colonna obbligatoria
<code>CHECK</code>	controlla una regola personalizzata
<code>DEFAULT</code>	assegna un valore se non lo specifichi

Un esempio completo

Questa tabella usa più vincoli insieme:

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  nome VARCHAR(100) NOT NULL,  
  eta INT CHECK (eta >= 0),  
  attivo BOOLEAN DEFAULT TRUE  
);
```

Leggiamola pezzo per pezzo:

- `id INT PRIMARY KEY` : ogni cliente ha un identificatore unico.
- `email VARCHAR(150) UNIQUE NOT NULL` : l'email è obbligatoria e non può essere duplicata.
- `nome VARCHAR(100) NOT NULL` : il nome è obbligatorio.
- `eta INT CHECK (eta >= 0)` : l'età non può essere negativa.
- `attivo BOOLEAN DEFAULT TRUE` : se non dici nulla, il cliente parte come attivo.

Perché sono utili

Senza vincoli, il database potrebbe accettare dati sbagliati:

- due utenti con la stessa email
- un prodotto con prezzo negativo
- un cliente senza nome
- un ordine collegato a un cliente che non esiste

I controlli nel codice dell'applicazione sono utili, ma non bastano sempre. Il database è l'ultimo posto in cui puoi fermare un dato sbagliato prima che venga salvato.

Un esempio con CHECK

`CHECK` ti permette di scrivere una regola semplice.

```
prezzo DECIMAL(10, 2) CHECK (prezzo >= 0)
```

Questa regola dice: il prezzo deve essere maggiore o uguale a zero.

Se qualcuno prova a inserire un prezzo negativo, il database rifiuta il dato.

Un esempio con DEFAULT

DEFAULT entra in gioco quando non specifichi un valore.

```
attivo BOOLEAN DEFAULT TRUE
```

Se inserisci un cliente senza dire se è attivo, il database userà **TRUE** .

È come dire: "se non ricevi istruzioni diverse, parti da questo valore".

Riepilogo rapido

- I vincoli sono regole applicate dal database.
- **PRIMARY KEY** identifica le righe.
- **FOREIGN KEY** protegge i collegamenti tra tabelle.
- **UNIQUE** evita duplicati.
- **NOT NULL** , **CHECK** e **DEFAULT** aiutano a mantenere dati più puliti.

Modellazione dati

Come progettare uno schema ordinato prima ancora di iniziare a riempirlo.

Prima di scrivere query, conviene disegnare la mappa

La modellazione dati è il momento in cui decidi come organizzare il mondo dentro il database. Prima di scrivere query, conviene fermarsi a capire quali oggetti esistono e come si collegano.

È come disegnare la pianta di una casa prima di iniziare i lavori.

Decidi:

- quali tabelle servono
- quali colonne deve avere ogni tabella
- come si collegano tra loro

:::note Uno schema è il progetto del database. Pensa allo schema come alla mappa dell'armadio: ti dice quali ripiani esistono e cosa va messo in ognuno. :::

Perché questa fase conta così tanto

Uno schema chiaro rende più semplice:

- scrivere query corrette
- evitare duplicazioni inutili
- capire dove deve vivere ogni dato
- far crescere il progetto senza caos

Un piccolo esempio mentale

Se stai costruendo un e-commerce, potresti identificare entità come:

- clienti
- prodotti
- ordini
- righe ordine

Ogni tabella dovrebbe rappresentare un concetto chiaro. Se una tabella prova a fare troppe cose insieme, di solito è un campanello d'allarme.

Una domanda pratica per iniziare

Quando non sai da dove partire, chiediti:

1. quali cose devo ricordare?

2. quali informazioni descrivono ogni cosa?
3. quali cose sono collegate tra loro?

Se stai progettando una biblioteca, le cose potrebbero essere `libri`, `lettori` e `prestiti`.

Un esempio piccolo

Questa tabella rappresenta un solo concetto: i clienti.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100),  
  email VARCHAR(150)  
);
```

La tabella non contiene ordini, pagamenti o prodotti. **Contiene solo informazioni sui clienti.**

Questa separazione rende il database più facile da capire e da far crescere.

Il segnale che qualcosa non va

Fermati se una colonna sembra contenere una lista intera, per esempio `prodotti_comprati`, oppure se ripeti sempre gli stessi dati in tante righe.

Di solito è il database che ti sta dicendo: **"forse qui serve una tabella in più"**.

Sequence e auto-increment

Come generare identificatori automatici senza sceglierli a mano.

Dare numeri a mano è scomodo e rischioso

Molte tabelle hanno una colonna `id` che cresce automaticamente. È come il numeretto preso in fila: ogni volta il sistema assegna il successivo.

Questo evita di dover scegliere gli identificatori a mano, con il rischio di errori o duplicati.

Un esempio semplice

In molti database puoi chiedere al sistema di creare automaticamente l'identificatore:

```
CREATE TABLE clienti (  
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    nome VARCHAR(100)  
);
```

Quando inserisci un cliente, non scrivi tu l' `id` :

```
INSERT INTO clienti (nome)  
VALUES ('Luca');
```

Il database assegna il prossimo numero disponibile.

I nomi cambiano a seconda del database:

- PostgreSQL: `IDENTITY` o `SERIAL`
- MySQL: `AUTO_INCREMENT`
- SQL Server: `IDENTITY`
- SQLite: `INTEGER PRIMARY KEY`

Perché è comodo

L'auto-incremento è utile quando:

- hai bisogno di una chiave primaria numerica
- vuoi evitare collisioni tra identificatori
- non vuoi preoccuparti di scegliere il prossimo numero

Una cosa importante da capire

Il numero automatico serve soprattutto come identificatore tecnico. Non sempre è una buona idea attribuirgli significati di business, come numero fattura o codice pubblico visibile all'utente.

Per quei casi, spesso servono regole diverse.

:::note Se vedi salti nei numeri, non è per forza un problema. Gli identificatori automatici servono a distinguere le righe, non a contare perfettamente senza buchi. :::

Relazioni tra tabelle

Uno a uno, uno a molti e molti a molti spiegati con esempi concreti.

Quando le tabelle si collegano

Un database diventa davvero utile quando le tabelle non restano isolate.

Pensa a una scuola. Hai studenti, classi, insegnanti e voti. Ogni elenco ha senso da solo, ma il valore vero arriva quando capisci come si collegano.

In SQL questi collegamenti si chiamano **relazioni**.

Le relazioni più comuni sono tre:

- uno a uno
- uno a molti
- molti a molti

Uno a uno

Una relazione uno a uno significa che una riga di una tabella corrisponde a una sola riga di un'altra tabella.

Esempio: un utente può avere un solo profilo esteso.

È come una persona e la sua carta d'identità: una persona ha una carta, e quella carta appartiene a quella persona.

```
CREATE TABLE utenti (  
    id INT PRIMARY KEY,  
    email VARCHAR(150) NOT NULL  
);  
  
CREATE TABLE profili (  
    utente_id INT PRIMARY KEY,  
    bio VARCHAR(500),  
    FOREIGN KEY (utente_id) REFERENCES utenti(id)  
);
```

Qui `profili.utente_id` identifica il profilo e, allo stesso tempo, punta all'utente collegato.

Uno a molti

Questa è una delle relazioni più comuni.

Un cliente può avere molti ordini. Ma ogni ordine appartiene a un solo cliente.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE ordini (  
  id INT PRIMARY KEY,  
  cliente_id INT NOT NULL,  
  totale DECIMAL(10, 2) NOT NULL,  
  FOREIGN KEY (cliente_id) REFERENCES clienti(id)  
);
```

La colonna `cliente_id` sta nella tabella `ordini`, perché ogni ordine deve ricordare a quale cliente appartiene.

È come una biblioteca: un lettore può fare molti prestiti, ma ogni prestito è intestato a un lettore preciso.

Molti a molti

Una relazione molti a molti significa che entrambe le parti possono collegarsi a più righe.

Per esempio:

- un ordine può contenere molti prodotti
- lo stesso prodotto può comparire in molti ordini

In questi casi si usa una **tabella ponte**.

```
CREATE TABLE righe_ordine (  
  ordine_id INT,  
  prodotto_id INT,  
  quantita INT,  
  PRIMARY KEY (ordine_id, prodotto_id),  
  FOREIGN KEY (ordine_id) REFERENCES ordini(id),  
  FOREIGN KEY (prodotto_id) REFERENCES prodotti(id)  
);
```

`righe_ordine` collega ordini e prodotti. In più salva anche la quantità.

È come un foglio di iscrizioni: non è solo la lista degli studenti e non è solo la lista dei corsi. Dice quale studente partecipa a quale corso.

Come capire che relazione ti serve

Fai la domanda in entrambe le direzioni.

Per esempio:

- un cliente può avere molti ordini?
- un ordine può appartenere a molti clienti?

Se “molti” vale da una parte sola, probabilmente hai una relazione uno a molti.

Se “molti” vale da entrambe le parti, probabilmente ti serve una tabella ponte.

Il ruolo delle chiavi esterne

Una **chiave esterna** (`FOREIGN KEY`) protegge il collegamento tra due tabelle.

Senza questa regola, potresti salvare un ordine con `cliente_id = 999` anche se non esiste nessun cliente con id 999.

Con la chiave esterna, il database controlla per te che il collegamento abbia senso.

Riepilogo rapido

- Uno a uno: una riga corrisponde a una sola riga.
- Uno a molti: una riga può collegarsi a molte righe.
- Molti a molti: di solito serve una tabella ponte.
- Le chiavi esterne aiutano il database a proteggere i collegamenti.

Proprietà ACID

Atomicità, coerenza, isolamento e durabilità spiegati senza formalismi inutili.

ACID è un modo rapido per dire: questa transazione è affidabile

ACID è una sigla usata per descrivere le proprietà attese da una buona transazione.

Pensa a una consegna importante. Vuoi che parta tutta, arrivi nel posto giusto, non si mescoli con altre consegne e resti registrata dopo la conferma.

- **Atomicità:** o tutto o niente
- **Coerenza:** i dati restano validi
- **Isolamento:** le transazioni non si disturbano troppo
- **Durabilità:** dopo il commit, il dato resta salvato

Un significato alla volta

Atomicità vuol dire che non vuoi mezze operazioni. Se trasferisci denaro, non vuoi togliere soldi da un conto senza aggiungerli all'altro.

Coerenza vuol dire che i dati devono rispettare le regole del database. Per esempio, un ordine non dovrebbe puntare a un cliente inesistente.

Isolamento vuol dire che più operazioni contemporanee non devono pestarsi troppo i piedi.

Durabilità vuol dire che, dopo la conferma, il risultato non deve sparire.

Perché dovresti ricordartelo

Non serve recitarlo come una poesia a memoria. Ti serve come bussola mentale.

Quando studi transazioni, concorrenza o affidabilità del database, **ACID ti ricorda che l'obiettivo non è solo eseguire query, ma eseguirle bene anche sotto stress.**

:::note ACID non è un comando SQL. È un modo per descrivere il comportamento affidabile che ti aspetti dalle transazioni. :::

Deadlock

Quando due transazioni si bloccano a vicenda e come capirlo.

Qui il problema non è il carico: è l'incastro sbagliato

Un deadlock succede quando due transazioni si aspettano a vicenda e nessuna può proseguire. È come due persone ferme in un corridoio stretto, ognuna in attesa che si sposti l'altra.

Un esempio mentale semplice

- la transazione A blocca la riga 1 e poi vuole la riga 2
- la transazione B blocca la riga 2 e poi vuole la riga 1

Risultato: si fermano entrambe.

Cosa fa il database

Molti database sanno rilevare questa situazione e annullano una delle due transazioni per sbloccare il sistema.

Questo non significa che il database "ha perso dati" a caso. Di solito annulla una transazione, segnala l'errore e lascia all'applicazione il compito di riprovare se ha senso.

Come ridurre il rischio

Puoi abbassare la probabilità di deadlock se:

- tieni le transazioni brevi
- accedi alle tabelle sempre nello stesso ordine
- eviti operazioni inutilmente lunghe mentre i lock sono attivi

Capire i deadlock è importante perché ti insegna che la concorrenza non è magia: è coordinazione.

Un'abitudine pratica

Se due parti del programma modificano sempre `clienti` e poi `ordini`, è meglio che tutte seguano quell'ordine.

Quando ogni pezzo del sistema prende le risorse nello stesso ordine, gli incastrici diventano meno probabili.

Livelli di isolamento

READ COMMITTED, REPEATABLE READ, SERIALIZABLE e gli altri livelli principali.

Quando più persone lavorano insieme, il problema non è solo cosa fai ma cosa vedi

I livelli di isolamento stabiliscono quanto una transazione può vedere del lavoro delle altre transazioni in corso.

Se due persone modificano o leggono dati nello stesso momento, il database deve decidere quanta "privacy" dare a ciascuna operazione.

Pensa a due persone che aggiornano lo stesso documento. Possono vedere subito le modifiche dell'altra? Devono aspettare? Il sistema deve scegliere una regola.

Livelli noti:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

La regola intuitiva

Più isolamento significa di solito più sicurezza e prevedibilità, ma anche meno libertà di lavorare in parallelo.

È sempre un equilibrio tra protezione e prestazioni.

Come pensarli senza perdersi

Non è indispensabile memorizzare subito i dettagli fini di ogni livello. All'inizio ti basta capire questo:

- i livelli bassi lasciano più libertà
- i livelli alti proteggono di più dai comportamenti strani

Una scala mentale utile

READ COMMITTED è una scelta comune: una transazione vede solo dati già confermati da altre transazioni.

REPEATABLE READ protegge di più la lettura: se rileggi gli stessi dati nella stessa transazione, ti aspetti più stabilità.

Il più rigoroso

SERIALIZABLE è il livello più severo tra quelli classici. Fa comportare le transazioni come se passassero una alla volta.

È molto protettivo, ma può essere più costoso in termini di concorrenza.

:::caution I dettagli precisi cambiano tra database. Prima impara l'idea generale, poi controlla il comportamento del database che stai usando. :::

Locking e concorrenza

Come il database gestisce accessi simultanei agli stessi dati senza farli scontrare.

Quando due mani cercano lo stesso oggetto nello stesso momento

La concorrenza nasce quando più utenti o processi lavorano sugli stessi dati nello stesso momento. Il database deve evitare che si creino conflitti o risultati incoerenti.

Un **lock** è come il cartello "occupato" appeso a una porta. Dice agli altri: aspetta un attimo, questa parte è già in uso.

In SQL, quel cartello può riguardare una riga, una tabella o altre parti interne del database.

Perché servono i lock

Senza meccanismi di locking, due aggiornamenti contemporanei potrebbero:

- sovrasciversi a vicenda
- leggere dati a metà
- produrre risultati difficili da prevedere

La cosa importante da capire

Il locking non è un difetto. È una protezione. Il problema nasce solo quando i lock durano troppo o vengono gestiti male.

Per questo conviene scrivere transazioni brevi e mirate, invece di lasciarle aperte inutilmente.

Un esempio mentale

Se due casse del supermercato provano a vendere l'ultimo pezzo dello stesso prodotto nello stesso momento, serve una regola. Il database deve evitare che entrambe dicano "venduto" come se il pezzo fosse doppio.

I lock aiutano a coordinare questi momenti delicati.

:::tip Quando una query resta bloccata, non pensare subito a un errore. Potrebbe essere in attesa che un'altra transazione finisca. :::

Transazioni

BEGIN, COMMIT e ROLLBACK per trattare più operazioni come un unico blocco.

Alcune operazioni devono riuscire insieme oppure fallire insieme

Una transazione è un gruppo di operazioni che il database tratta come un solo blocco logico. Questo è fondamentale quando uno stato "a metà" sarebbe pericoloso o incoerente.

Pensa a un trasferimento di denaro: togliere soldi da un conto e aggiungerli a un altro deve funzionare tutto insieme.

Un esempio concreto

```
BEGIN;  
UPDATE conti  
SET saldo = saldo - 100  
WHERE id = 1;  
  
UPDATE conti  
SET saldo = saldo + 100  
WHERE id = 2;  
  
COMMIT;
```

Qui il lavoro inizia con `BEGIN` e si conclude con `COMMIT`.

Il ruolo dei tre comandi chiave

- `BEGIN` : inizia la transazione
- `COMMIT` : conferma tutto in modo definitivo
- `ROLLBACK` : annulla le modifiche della transazione

Quando entra in gioco **ROLLBACK**

Se qualcosa va storto a metà, puoi fare marcia indietro:

```
ROLLBACK;
```

Questa è la grande forza delle transazioni: ti aiutano a evitare stati rotti o incompleti.

Come pensarla nella vita reale

Immagina di spedire un pacco. Non basta stampare l'etichetta: devi anche pagare la spedizione e consegnarlo al corriere.

Se uno di questi passaggi fallisce, non vuoi lasciare tutto in uno stato confuso. Vuoi tornare indietro e riprovare con ordine.

:::tip Tieni le transazioni brevi. Più restano aperte, più possono bloccare il lavoro di altre operazioni. :::

CREATE VIEW

Come salvare una query con un nome e riusarla come se fosse una tabella.

Una view è una finestra salvata sui dati

Una view è una query salvata con un nome. Invece di riscrivere ogni volta la stessa richiesta, puoi salvarla e riutilizzarla.

Pensa a una finestra affacciata su un cortile. La finestra non è il cortile. Ti mostra solo una parte del cortile, sempre dallo stesso punto di vista.

Una view funziona così: non inventa nuovi dati, ma mostra i dati attraverso una query già preparata.

Un esempio piccolo

```
CREATE VIEW clienti_attivi AS
SELECT id, nome, email
FROM clienti
WHERE attivo = TRUE;
```

Da quel momento puoi leggere `clienti_attivi` come se fosse una tabella:

```
SELECT nome, email
FROM clienti_attivi;
```

Perché è comoda

Le view aiutano quando:

- una query viene usata spesso
- vuoi nascondere complessità dietro un nome più semplice
- vuoi offrire una vista più pulita di dati più grandi

Cosa NON è

Una view, di base, non è una copia indipendente dei dati. **È più simile a una finestra su una query.**

Quando guardi dentro, vedi il risultato di quella definizione.

:::note Se i dati originali cambiano, anche il risultato della view cambia. La view normale non conserva una fotografia separata. :::

Materialized view

Una vista che salva anche il risultato per rendere certe letture più veloci.

Qui non salvi solo la ricetta: salvi anche il piatto pronto

Una materialized view è simile a una view, ma con una differenza importante: salva anche il risultato della query.

È come avere una fotografia già pronta, invece di rifare il calcolo ogni volta da zero.

Una view normale ricalcola la scena quando guardi. Una materialized view conserva una copia del risultato, così alcune letture possono essere molto più veloci.

Perché può essere utile

Se una query è pesante e viene letta spesso, avere il risultato già materializzato può velocizzare molto le consultazioni.

Questo succede spesso in report e dashboard.

Un esempio piccolo

Immagina un report che somma gli ordini per giorno:

```
CREATE MATERIALIZED VIEW vendite_giornaliere AS
SELECT data_ordine, SUM(totale) AS totale_giorno
FROM ordini
GROUP BY data_ordine;
```

Invece di rifare la somma ogni volta, il database può leggere il risultato già preparato.

Il punto da ricordare

La materialized view non si aggiorna sempre da sola nello stesso modo di una view normale. **Ogni tanto il risultato va rigenerato o aggiornato.**

:::note Non tutti i database la supportano nello stesso modo. :::

Quando può avere senso sceglierla

È una buona candidata quando il risultato è costoso da calcolare ma viene letto spesso.

Per report e dashboard, questa differenza può essere molto importante.

:::caution Se i dati cambiano spesso, chiediti quanto puoi accettare un risultato non aggiornato al secondo. :::